

Bachelor Thesis

Fachhochschule Dortmund
University of Applied Sciences and Arts

**Predictive Maintenance for Turbofan Engines:
Remaining Useful Life Prediction on the NASA C-MAPSS Dataset**

Ali Mohamed Ali Mohamed Elshafey

First Examiner: Prof. Dr. Nadja Bauer

Second Examiner: Prof. Dr. Klaus Kaiser

Department of Computer Science

Submission Date: September 22, 2026

Table of Contents

Abstract	i
Kurzfassung	ii
List of Figures and Tables	iii
List of Abbreviations	iv
List of Symbols	v
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 Objectives and Research Questions	2
1.4 Scope and Delimitation	3
2 Theoretical Background	5
2.1 Predictive Maintenance and Remaining Useful Life	5
2.2 Machine Learning Fundamentals	5
2.3 Definition of the RUL Target	7
2.4 Models	8
2.4.1 Linear Regression	8
2.4.2 Extreme Gradient Boosting (XGBoost)	9
2.4.3 Long Short-Term Memory (LSTM)	11
2.4.4 Chronos-2	13
2.5 Evaluation Metrics	15
2.6 Statistical Comparison	17
3 Related Work	18
3.1 Scope of the Review	18
3.2 Recurrent and Convolutional Approaches	18
3.3 Transformer-Based Approaches	19
3.4 Simple and Interpretable Models	20
3.5 The Role of Target Definition and Input Representation	20
3.6 Positioning of the Present Work	21
4 The C-MAPSS Dataset	23
4.1 The C-MAPSS Simulator and the Turbofan Engine	23
4.2 Dataset Structure and Feature Categories	24
4.3 Operating Condition and Fault Mode	25
4.4 Sensors and Operational Settings	26

4.5 Remaining Useful Life Labels	27
5 Exploratory Data Analysis.....	28
5.1 Analytical Scope and Data Integrity.....	28
5.2 Engine Lifetime Distribution.....	28
5.3 Variable Screening and Redundancy	29
5.4 Sensor Behaviour over an Engine's Life.....	31
5.4.1 Trend Direction and Its Consistency	32
5.4.2 Non-Linear Shape.....	33
5.4.3 Between-Engine Variation	34
5.5 Properties of the Recorded Signals.....	35
5.6 Detectability of an Early-Life Signal	37
5.7 Target Definition and Modelling Implications	39
6 Methodology.....	41
6.1 The Prediction Task and Data Preparation	41
6.2 Experimental Protocol	42
6.2.1 Validation Design.....	42
6.2.2 Separation of the Training and Test Data	43
6.3 Definition of the Target	44
6.4 Representation of the Observed History	45
6.4.1 Equalising the Observation Horizon	45
6.4.2 Summary Features for the Tabular Models	46
6.4.3 Sequence Representation for the LSTM and Chronos-2	47
6.4.4 Additional Representations Examined	47
6.5 Final Configuration and Evaluation Protocol	49
6.5.1 Smoothing of the Sensor Signals	49
6.5.2 Model Configurations.....	50
6.5.3 Evaluation Protocol.....	50
6.6 Design Decisions and Final Configuration.....	51
7 Results and Discussion	53
7.1 Effect of the History Representation.....	53
7.2 Comparative Predictive Performance.....	53
7.3 Computational Cost and Inspectability	56
7.4 The Target Definition and the Output Constraint	57
7.5 Post-hoc Inspection and the Behaviour of Sensor s9	59
7.6 Comparison with Published Work	61
7.7 Answers to the Research Questions	61
7.8 Limitations and the Scope of the Conclusions	62

8 Conclusion and Outlook..... 64

8.1 Contributions 64

8.2 Future Work 64

Appendix A Sensor and Operational-Setting Reference 66

Appendix B Data-Quality and Data-Separation Checks 68

References..... 70

Abstract

Unplanned engine failure is expensive and, in service, potentially unsafe. Predictive maintenance answers this by estimating a component's Remaining Useful Life from condition-monitoring data. On the C-MAPSS turbofan benchmark, published work reports low errors for increasingly complex architectures, but the studies differ in how much history each model receives, in how the target is built, and in which metrics they report. That leaves open how much of the progress belongs to the architectures themselves.

This thesis compares four pipelines on the FD001 subset under one protocol: linear regression, XGBoost, a Long Short-Term Memory network, and a frozen Chronos-2 representation with a trained regression head. Each receives the same observation horizon of every engine, in the form its architecture accepts, so the amount of history supplied does not confound the comparison. Every design decision was settled on the training engines, and the pipelines were frozen before the test engines were used; an earlier round that had consulted the test set is disclosed in Section 6.2.2. Performance is reported with the mean absolute error, the root mean squared error and the asymmetric score of the 2008 Prognostics and Health Management (PHM) Data Challenge, together with fitting time and model size.

Linear regression gave the largest error on all three metrics, and its engine-level errors differed from those of every other pipeline after correction for multiple comparisons. Among XGBoost, the LSTM and Chronos-2 no corrected difference was found; their mean errors lie within 1.12 cycles of one another. XGBoost was fitted on a CPU in under half a minute, while both sequence pipelines needed a GPU. A factorial experiment separated the training labels from the constraint applied to the predictions: the effect of changing the labels was more than four times larger without the constraint than with it, so an improvement credited to a target definition may partly belong to the post-processing that came with it.

The results support moving beyond a single global linear model on this benchmark, but give no evidence that either sequence architecture improves on a gradient-boosted tree ensemble once the observed history is aligned. The comparison holds complete pipelines constant rather than architectures, and covers one subset under one protocol.

Keywords: predictive maintenance; remaining useful life; C-MAPSS; turbofan engine; XGBoost; long short-term memory; time-series foundation model; prognostics

Kurzfassung

Der unerwartete Ausfall eines Flugzeugtriebwerks verursacht hohe Kosten und kann im Betrieb sicherheitskritisch werden. Die vorausschauende Instandhaltung begegnet diesem Problem, indem sie aus Zustandsdaten die verbleibende Nutzungsdauer einer Komponente schätzt. Für den Turbofan-Benchmark C-MAPSS berichtet die Literatur niedrige Fehler für zunehmend komplexe Architekturen. Die Studien unterscheiden sich jedoch darin, wie viel der Betriebshistorie ein Modell erhält, wie die Zielgröße konstruiert wird und welche Metriken berichtet werden. Wie viel des berichteten Fortschritts tatsächlich auf die Architekturen selbst zurückgeht, bleibt daher offen.

Die vorliegende Arbeit vergleicht vier Modellierungspipelines auf dem Teildatensatz FD001 unter einem einheitlichen Protokoll: eine lineare Regression, XGBoost, ein Long-Short-Term-Memory-Netz sowie eine eingefrorene Chronos-2-Repräsentation mit trainiertem Regressionskopf. Jede Pipeline erhält Messwerte über denselben beobachteten Abschnitt jedes Triebwerks, jeweils in der Form, die ihre Architektur verarbeitet. Unterschiede im Umfang der bereitgestellten Historie lassen sich dadurch nicht mit Unterschieden zwischen den Modellen verwechseln. Die Entscheidungen über Zieldefinition, Repräsentation der Historie und Vorverarbeitung wurden in einer fünffachen Kreuzvalidierung auf Triebwerksebene innerhalb der Trainingsdaten getroffen, die Modellkonfigurationen auf einem vorgelagerten Holdout auf Triebwerksebene. Anschließend wurden die Pipelines eingefroren, bevor die bestätigende Auswertung auf den offiziellen Testtriebwerken erfolgte; die frühere sowie die nachgelagerte Verwendung dieser Daten sind in Abschnitt 6.2.2 offengelegt. Berichtet werden der mittlere absolute Fehler, die Wurzel des mittleren quadratischen Fehlers und die asymmetrische Bewertungsfunktion der PHM-Data-Challenge 2008, ergänzt um gemessene Trainingszeiten und Modellgrößen.

Die lineare Regression wies auf allen drei Metriken den größten Fehler auf; ihre triebwerksweisen Fehler unterschieden sich nach Korrektur für multiples Testen von denen jeder anderen Pipeline. Zwischen XGBoost, dem LSTM und Chronos-2 ließ sich dagegen kein korrigierter Unterschied im absoluten Fehler nachweisen; ihre mittleren Fehler liegen innerhalb von 1,12 Zyklen. XGBoost wurde dabei in weniger als einer halben Minute auf einer CPU angepasst, während beide Sequenzpipelines auf einer GPU liefen. Ein faktorielles Experiment trennte die Wirkung der Trainingslabels von der Wirkung der auf die Vorhersagen angewandten Beschränkung: Der Effekt der Labels war ohne die Beschränkung mehr als viermal so groß wie mit ihr. Eine Verbesserung, die einer Zieldefinition zugeschrieben wird, kann daher teilweise auf die begleitende Nachverarbeitung entfallen.

Die Ergebnisse sprechen dafür, auf diesem Benchmark über ein einzelnes globales lineares Modell hinauszugehen. Sie liefern jedoch keinen Beleg dafür, dass eine der beiden Sequenzarchitekturen ein gradientenverstärktes Baumverfahren übertrifft, sobald die beobachtete Historie angeglichen ist. Da der Vergleich vollständige Pipelines und nicht einzelne Architekturen konstant hält und da er einen Teildatensatz unter einem Protokoll abdeckt, sind die Schlussfolgerungen entsprechend begrenzt.

List of Figures and Tables

Figures

Figure 2.1	Additive prediction of remaining useful life by two regression trees	10
Figure 2.2	Structure of a standard forget-gate LSTM cell	12
Figure 2.3	The Chronos-2 inference pipeline	13
Figure 2.4	The PHM score as a function of prediction error	16
Figure 4.1	Simplified diagram of the turbofan engine modelled by C-MAPSS	23
Figure 4.2	Recorded values of four sensors against operating cycle	26
Figure 5.1	Distribution of observed lifetimes across the 100 training engines	29
Figure 5.2	Pearson correlation matrix of the FD001 training data	31
Figure 5.3	Consistency of sensor trends across the 100 training engines	32
Figure 5.4	Median standardised deviation of each sensor from its starting level	34
Figure 5.5	Between-engine standard deviation at failure relative to start	35
Figure 5.6	All recorded values of sensor s8 across the 100 training engines	37
Figure 5.7	Predicting total lifetime from the first N cycles, by window length	38
Figure 7.1	Performance of the four frozen pipelines on the 100 test engines	54
Figure 7.2	Post-hoc permutation importance for the XGBoost pipeline	60
Figure A.1	Mean recorded value of each of the twenty-one sensors	67

Tables

Table 3.1	Selected results relevant to the FD001 comparison	21
Table 4.1	The three categories of columns in the FD001 dataset	24
Table 4.2	Opening cycles of two training engines	25
Table 5.1	Variability screening of the twenty-four candidate variables	30
Table 5.2	Distribution of the training target under both definitions	39
Table 5.3	Findings of the exploratory analysis and the decisions they inform	39
Table 6.1	Validation mean absolute error under three levels of observed history	46
Table 6.2	Validation mean absolute error with and without the shape features	48
Table 6.3	Validation mean absolute error under four smoothing settings	49
Table 6.4	Model configurations and structural size	50
Table 6.5	Experiments and checks, and the decisions arising from them	51
Table 6.6	Final configuration of the four models	52
Table 7.1	Test performance of the four modelling pipelines	53
Table 7.2	Pairwise Wilcoxon signed-rank tests on engine-level absolute errors	54
Table 7.3	Measured computational cost	56
Table 7.4	Test performance under both training targets and both output settings	57
Table 7.5	Importance of each summary statistic	59
Table 7.6	The six highest-ranked sensor groups	59
Table 7.7	Limitations and their consequences	62
Table A.1	Physical description of the operational settings and sensors	66
Table B.1	Integrity checks and resulting decisions	68
Table B.2	Points of exposure between training and test data	68

List of Abbreviations

Abbreviation	Meaning
BiLSTM	Bidirectional Long Short-Term Memory
C-MAPSS	Commercial Modular Aero-Propulsion System Simulation
CPU	Central Processing Unit
DCNN	Deep Convolutional Neural Network
DSFormer	Dual-Scale Transformer
FD001	First C-MAPSS subset: one operating condition, one fault mode
GCU	Gated Convolutional Unit
GPU	Graphics Processing Unit
HPC	High-Pressure Compressor
HPT	High-Pressure Turbine
IQR	Interquartile Range
LDA	Linear Discriminant Analysis
LPC	Low-Pressure Compressor
LPT	Low-Pressure Turbine
LR	Linear Regression
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
N-CMAPSS	New C-MAPSS dataset, generated under real flight conditions
NASA	National Aeronautics and Space Administration
PHM	Prognostics and Health Management
RMSE	Root Mean Squared Error
RSS	Residual Sum of Squares
RUL	Remaining Useful Life
SD	Standard Deviation
SQ1 to SQ5	Subsidiary research questions, defined in Section 1.3
XGBoost	Extreme Gradient Boosting

List of Symbols

Symbol	Meaning
$c_1, \dots, c_n$	Recorded values of one sensor over an observed history of n cycles
E	Error of a fitted model on the data as recorded
$E_{p(j)}$	Error of the same model after input j has been permuted
$R_{\max}$	Upper bound applied to the piecewise target
L	Total lifetime of an engine, in cycles
n	Number of cycles in an observed history
N_c	Physical core speed, recorded by sensor s9
NR_c	Corrected core speed, recorded by sensor s14
r	Pearson correlation coefficient
ρ	Spearman rank correlation coefficient
R^2	Coefficient of determination
t	Cycle index within an engine's record
T_2	Total temperature at the fan inlet, recorded by sensor s1
T_{ref}	Reference temperature, 518.67 °R or 288.15 K
S	PHM score, summed over the test engines
s_i	Contribution of test engine i to the PHM score
d_i	Difference between predicted and true remaining life for engine i
y_i	True remaining useful life of test engine i
$\hat{y}_i$	Predicted remaining useful life of test engine i
N	Number of test engines

1 Introduction

1.1 Motivation

Modern industrial systems depend on complex machines whose unexpected failure can cause significant costs, operational downtime, and safety risks [1], [2]. Aircraft engines are a representative example: they operate under extreme thermal and mechanical loads, and a failure in service is both expensive and potentially dangerous [3]. For such systems, the central maintenance question is not only whether a component is currently working, but how much longer it can be expected to work before it fails. The quantity that answers this question is the Remaining Useful Life (RUL), defined as the number of operating cycles remaining before a system can no longer perform its intended function [3].

Maintenance strategies differ in how they address that question, and the strategy of interest here is predictive maintenance, which estimates the future condition of a component from monitoring data and schedules an intervention shortly before failure becomes likely [1]. The alternatives, and the position of predictive maintenance among them, are set out in Section 2.1. What makes the approach attractive is also what makes it difficult: its effectiveness rests entirely on translating sensor measurements, which describe the present state of a machine, into a statement about its future behaviour. That translation is a data-driven task, and it has become a central application area for machine learning in industrial engineering [2].

The choice of topic is also a personal one. The author's background combines earlier studies in mechanical engineering with current studies in data science, and predictive maintenance sits at the intersection of the two: it requires an understanding of how machines physically degrade on one side, and of the methods that detect that degradation in data on the other. Working on this problem allows the engineering context to be read alongside the statistical treatment, rather than treating the measurements in isolation from the system that produced them.

1.2 Problem Statement

Predicting the Remaining Useful Life of a degrading machine differs from many other supervised learning tasks in four respects.

The target is never observed during operation. A sensor can report a temperature or a pressure at the current cycle, but the true remaining life of an engine is known with certainty only once it has failed. The data used here reflect this directly: training engines are observed until failure, whereas test engines are observed only up to an earlier cycle, with their true remaining life supplied separately for evaluation [3] (Section 4.5). A model must therefore learn the relationship between sensor behaviour and remaining life from complete trajectories and then apply it to engines whose most severe degradation has not yet occurred.

The relationship between sensor behaviour and remaining life is neither simple nor identical across machines. Degradation develops gradually and becomes visible in the sensor signals only after an engine has operated for some time [4], and individual engines differ in their initial condition and in how quickly they deteriorate [3]. The same elapsed operating time can therefore correspond to very different remaining lives, so the degradation state has to be inferred

from the pattern of the measurements rather than from age alone. How pronounced these differences are in the data used here is examined in Chapter 5.

The target itself is constructed rather than measured. The most direct construction lets remaining life decrease from the very first cycle, which implies that a healthy engine already carries information about how long it will ultimately last. An alternative holds the target constant while the engine is still healthy and allows it to fall only once degradation becomes detectable [4]. The two definitions ask a model to learn different things; the argument behind them is developed in Section 2.3, and both are examined in this thesis.

The range of available methods is wide, from linear models to deep networks and, more recently, large pretrained time-series models. These differ not only in representational capacity but in computational cost, implementation effort, and data requirements. Published work reports strong accuracy for increasingly complex models, while reviews point to trade-offs among complexity, cost, precision, and practical applicability [5]. Whether the added complexity is repaid on a simplified single-condition benchmark of this kind is therefore an open question, and it is the one this thesis takes up.

1.3 Objectives and Research Questions

The objective is to compare four modelling pipelines for remaining-life prediction on a single benchmark subset, under a shared evaluation protocol in which each has access to the same observation horizon of an engine's history. A pipeline here is the complete chain from the recorded sensor values to a prediction, the preprocessing, the representation of the engine's history, the model itself, and any constraint applied to its output, and not the architecture alone. The aim is not to improve on the lowest published error. As Section 1.2 set out, a model given more history than another may perform better for that reason alone, and the design removes that confounding factor. It does not isolate architecture: the four pipelines still differ in representation, fitting procedure, and the number of engines contributing to parameter fitting, and those differences are stated rather than left implicit.

The overall question is therefore:

In the single-condition C-MAPSS FD001 setting, how do linear regression, XGBoost, an LSTM, and a frozen Chronos-2 representation compare in remaining-life prediction and computational cost when each has access to the same observation horizon of engine history?

The Chronos-2 backbone is used frozen: its weights are left exactly as its authors released them, and only a small regression head is fitted to the turbofan data (Section 2.4.4). The restriction to FD001 is explained in Section 1.4. Five subsidiary questions follow.

SQ1. How does the amount and form of the observed history supplied to a model affect its predictive performance? Two controlled validation experiments vary what a model receives while holding the model itself fixed (Section 6.4.1).

SQ2. Under a shared evaluation protocol and a matched observation horizon, do the four modelling pipelines differ in predictive accuracy? This is the main comparative question, assessed on the official test engines with paired significance tests. Because representation and

fitting procedure are not identical across the four, it concerns complete pipelines rather than architecture in isolation.

SQ3. What computational cost does each approach incur, measured by fitting time, model size, and the hardware on which each was run? Two approaches of comparable accuracy may differ by orders of magnitude in what they cost. Timings are reported with the hardware on which they were obtained.

SQ4. How does the target definition alter the learning problem and the resulting predictive performance? The target is constructed rather than supplied by the data, and its definition determines which differences a model is asked to learn.

SQ5. Which properties of the sensor data constrain the modelling choices, and what do they indicate about the problem itself? Several later decisions follow from how the measurements behave rather than from free choice.

These questions are framed descriptively. SQ2 asks whether the pipelines differ in predictive accuracy without presupposing an ordering between them, and without treating any observed difference as the effect of architecture alone.

1.4 Scope and Delimitation

The scope is limited deliberately, so that the research questions can be answered rigorously within the constraints of a bachelor's thesis.

Dataset. Only the FD001 subset of C-MAPSS is used. It is the simplest of the four subsets, with a single operating condition and a single fault mode [3], and the overall question concerns precisely such a simplified benchmark. Working with one subset also allows its sensors and degradation behaviour to be analysed in the depth that a sound interpretation of the results requires, rather than spreading the analysis across four. The structural differences between the subsets are described in Section 4.3, and the more complex ones are left to future work.

Models. Four models are compared, one from each of four families of increasing complexity: a linear baseline, a gradient-boosted tree model, a recurrent network, and a pretrained Transformer-based time-series foundation model. The work does not survey all available methods; the number is kept small so that the comparison remains controlled and interpretable. The tuning procedure is specified in Section 6.5.2, and the use made of the test set in Section 6.2.2.

Prediction task. Remaining Useful Life is estimated as a regression task, producing a numerical estimate of remaining cycles. Related prognostic tasks, such as classifying an engine as healthy or degraded, detecting fault onset, or identifying a fault type, are not addressed; with a single fault mode, fault classification would not be meaningful for this subset in any case.

Evaluation. Models are evaluated with established error metrics together with a measure of computational cost, following the standard protocol associated with the dataset (Section 2.5). Predicting remaining life continuously at every cycle of the test trajectories is not attempted, so that the comparison stays consistent with the established benchmark and the published literature.

Out of scope. Several topics related to predictive maintenance but not required by the research questions are excluded: the economic modelling of maintenance decisions, deployment in a real-time or embedded environment, transfer of a trained model to another engine type, and the analysis of real rather than simulated sensor data. They are taken up as directions for future work in Chapter 8.

2 Theoretical Background

2.1 Predictive Maintenance and Remaining Useful Life

Industrial operations are under continuous pressure to keep equipment reliable while reducing unplanned downtime and operating costs [1], [18]. The literature commonly distinguishes four maintenance strategies [20]. Reactive maintenance repairs a component only after it has failed, which avoids planning but exposes the operator to sudden interruptions. Preventive maintenance follows a fixed schedule based on time in service rather than on the state of the asset [18]. Proactive maintenance analyses data to identify and remove the underlying causes of degradation [20]. Predictive maintenance anticipates failure by studying operational data and monitoring the asset's condition [20]. It is closely related to condition-based maintenance and prognostics, in that it schedules an intervention on the predicted future condition of an asset rather than on a fixed timetable [1], [19], and industrial case studies report gains in cost, availability, and safety from doing so [2].

The central task within prognostics is estimating the Remaining Useful Life, introduced in Section 1.1. It need not be expressed in clock time: in the turbofan setting studied here it is measured in flight cycles, where each row of data is a snapshot from one cycle and one cycle corresponds to one flight [3], [19]. An accurate estimate allows an intervention to be timed so that neither useful component life is wasted nor failure occurs first [18].

2.2 Machine Learning Fundamentals

Machine learning extracts relationships from data rather than specifying them by hand. In supervised learning, every observation pairs one or more input measurements with an associated outcome, and the aim is to learn a mapping from the inputs to that outcome so that it can be predicted for observations not seen during learning [21]. Remaining-life prediction belongs to this family, since each historical observation can be paired with the remaining life subsequently realised [19]. The input measurements are called features and the quantity predicted the target; both terms are used throughout this thesis [21].

Formally, let X denote the vector of p features and Y a quantitative target. One assumes a relationship of the general form

$$Y = f(X) + \varepsilon,$$

where f is a fixed but unknown function of X and ε is a random error term that is independent of X and has mean zero [21]. The error term collects everything the features cannot explain, including measurement noise and unmeasured influences, and because no estimate of f can account for it, its variance sets a lower bound on the prediction error, known as the irreducible error [21]. The methods described in Section 2.4 differ principally in the form they assume for f and in how they estimate it.

A model that receives a recorded history rather than a single set of measurements needs that history reduced to a fixed number of inputs. Writing $c_1, \dots, c_n$ for the values one sensor has taken over an observed history of n cycles, and $t=1, \dots, n$ for the corresponding cycle positions, the quantities used for that purpose in this thesis are defined here. The mean and the standard

deviation carry their usual meaning and are not restated; the standard deviation is taken in its population form.

The slope is the coefficient of a straight line fitted to the observed values by least squares, with cycle position as the independent variable,

$$\text{slope} = \frac{\sum_{i=1}^n (t_i - \bar{t})(c_i - \bar{c})}{\sum_{i=1}^n (t_i - \bar{t})^2}.$$

It measures the average rate and direction of change over the observed history, in sensor units per cycle, and is undefined for a single observation. The last value is the most recent observation, c_n , and the delta is the difference between the most recent and the first observed value,

$$\text{delta} = c_n - c_1.$$

The delta measures the net displacement of the signal from where the record began and equals zero for a single observation. These five quantities, the mean, the standard deviation, the slope, the last value and the delta, are referred to throughout this thesis as the summary statistics of a sensor, and the model inputs they produce as the summary features.

Two further quantities describe the shape of a trajectory rather than its level or its overall rate. The quadratic coefficient is the coefficient of the squared term in a second-degree polynomial fitted to the observed values, with cycle position rescaled to the interval from zero to one before fitting so that the coefficient is comparable across histories of different lengths, which a fit in raw cycle units would not be; it is defined only where at least three observations are available. The slope ratio is the ratio of the slope over the most recent third of the observed history to the slope over its earliest third,

$$\text{slope ratio} = \frac{\text{slope over the most recent third}}{\text{slope over the earliest third}}.$$

A slope ratio above one indicates that the signal has changed faster recently than it did at the start of the record. The two thirds are thirds of the observed history, not of the engine's eventual life, so the most recent third of what has been recorded may lie well before the final third of the trajectory an engine will ultimately follow.

What matters is not how well a model reproduces the data it was fitted on but how it performs on observations it has never encountered. The data are therefore separated into a training set, used to estimate the model, and a test set, held back for the final assessment; where a method requires configuration choices, a third portion, the validation set, is used for those so that the test set remains untouched [21]. The two errors behave differently: as a model is made more flexible its training error falls steadily, while its test error often falls at first and then begins to rise [21]. Such a rise indicates overfitting, and it is why a more flexible model is not automatically a better one. This bears directly on the present work, since the four models compared differ substantially in flexibility.

A single division of this kind uses only part of the available data for each purpose, and the estimate it produces depends on which observations happened to fall on each side of the split. Cross-validation reduces that dependence: the data are divided into k parts, the model is fitted

k times, and a different part is held out on each occasion, so that every observation is used for assessment exactly once [21]. Where the observations are not independent of one another, the division has to respect that structure rather than treat individual rows as interchangeable. What this means for engine data, in which consecutive cycles describe the same trajectory, is set out in Section 6.2.

Some fitting procedures make random choices, for example when initialising the parameters of a network or subsampling the data. A seed is the number that fixes those choices, so that the same run can be repeated.

2.3 Definition of the RUL Target

Sensor recordings describe an engine's condition at a given moment but do not state how much operational life remains. The target must therefore be constructed rather than read from the data [3], which makes its definition a modelling decision in its own right.

The most direct construction assigns to each cycle the number of cycles remaining until failure. For an engine whose complete run lasts L cycles, the target at cycle t is

$$\text{RUL}_{\text{linear}}(t) = L - t, \quad \text{for } t = 1, \dots, L, \quad (2.1)$$

where L is the length of the engine's recorded run and t the current cycle. The label decreases by one at every cycle from the very first one onward. This definition requires no additional assumptions, but it carries an implicit claim: that an engine still operating in a healthy state already exhibits, in its sensor readings, information about how long it will ultimately last.

Heimes questioned that claim [4]. He observed that degradation is generally not noticeable until a unit has been operated for some period and an initial fault has developed and asked how one could expect to predict how long an item will last when it is brand new and in a very healthy state [4]. Under the linear definition a model is nevertheless required to distinguish a target of, say, 300 from one of 250 during a phase in which the sensor signals of the two engines are practically indistinguishable, so it is penalised for failing at a task the data do not support.

The alternative he proposed holds the target at a constant value during the assumed healthy phase and allows it to decrease linearly thereafter [4]. Writing R_{max} for that constant, the target becomes

$$\text{RUL}_{\text{piecewise}}(t) = \min(L - t, R_{\text{max}}), \quad \text{for } t = 1, \dots, L, \quad (2.2)$$

so that the label is flat while more than R_{max} cycles remain and identical to the linear definition afterwards. The construction does not inspect the sensor signals to find where degradation begins: it assumes that any cycle with more than R_{max} cycles remaining belongs to the healthy phase, so the transition occurs at a fixed threshold rather than at an engine-specific onset. It has since become standard in the C-MAPSS literature [6].

The choice of R_{max} is not arbitrary. Heimes derived his from the length of the runs in the challenge data, the early steady-state output of his network, and the fact that overestimating remaining life was penalised more heavily than underestimating it, adopting 130 after experimentation [4]. The constant is bounded below by what it is intended to achieve: to alter at least one label in every trajectory it must fall below the largest target of the shortest engine.

Heimes reported that outputs became less noisy once the target was capped, while the shape near failure was largely unaffected [4].

Capping and clipping are distinct operations and are easily conflated. Capping is applied to the training labels: it changes what the model is asked to learn, and a model fitted on capped labels has never been shown a target above $R_{\max}$. Clipping is applied to a prediction after fitting: it restricts the output to a chosen range without altering the model or anything it learned. The two can be varied independently, and Section 7.4 reports what each contributes.

Both definitions are examined in this thesis. The value of $R_{\max}$ adopted here, and the reasoning behind it, are given in Section 6.3.

2.4 Models

The four models are presented in order of increasing representational flexibility, meaning the range of relationships each can express between the features and the target. This ordering does not follow model size: the largest by stored parameters is used with only a small trained component, so computational cost is reported separately (Section 2.4.4). Each model is described through the components it consists of, its main equation, the properties that matter for remaining-life prediction, and the way it is used in this work; full derivations remain in the cited sources.

One property separates the four into two groups and recurs throughout the comparison. Linear regression and XGBoost receive each observation as an unordered feature vector and have no internal representation of sequence order, so any information about how a signal has evolved must be encoded into the features beforehand. The LSTM and Chronos-2 receive the ordered measurements themselves. The two groups therefore differ not only in architecture but in the form in which the same operating history reaches them, which is the distinction SQ1 addresses.

2.4.1 Linear Regression

Linear regression models a quantitative target as a weighted sum of the features. It assumes that the effect of each feature on the target is fixed in size and direction and does not depend on the values of the other features [21].

The model consists of three components: one coefficient for each of the p features, a single intercept, and an error term collecting what the features cannot explain. Fitting it means choosing values for the $p + 1$ coefficients under a stated criterion.

Using the feature vector X defined in Section 2.2, the model is

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon,$$

where Y is the remaining useful life to be predicted, X_1 to X_p are the features describing an engine at the prediction point, β_0 is the intercept, each β_j is the average change in Y associated with a one-unit increase in X_j while the remaining features are held fixed, and ε is the irreducible error introduced in Section 2.2. A prediction for a new engine is formed from the fitted coefficients without the error term.

The coefficients are estimated by least squares, which selects the values minimising the residual sum of squares

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2,$$

where n is the number of training observations, y_i is the observed remaining life of observation i , and x_{ij} is the value of feature j for that observation. The minimising coefficients are obtained directly rather than by iterative search [21]. The fitted equation can be read instead of being treated as a black box, since each coefficient carries an explicit meaning within the model, and in its unregularised form least squares introduces no hyperparameters, so the baseline is reproducible without a tuning procedure.

Ridge regression adds to the residual sum of squares a penalty proportional to the sum of the squared coefficients, so that the fit is balanced against the size of the coefficients. The strength of that penalty is set by a single parameter: at zero the method reduces to ordinary least squares, and as it grows the coefficients are shrunk towards zero. Shrinking them stabilises the fit where features are correlated, at the cost of introducing a parameter that must be chosen [21, Ch. 6]. Ridge regression is not one of the four compared pipelines; it is used only as a diagnostic predictor in Sections 5.6 and 6.4.4.

The model's fixed functional shape cannot represent a relationship that curves or reverses [21]. Collinearity adds a second difficulty: sensor signals that rise and fall together admit many coefficient combinations fitting the data almost equally well, so the individual coefficients become unstable and cannot be read as measures of feature importance, even where the predictions remain adequate [21].

In this thesis linear regression is applied to the summary features of Section 6.4.2, and its error establishes the level against which the three more flexible pipelines are judged. A full treatment is given in [21, Ch. 3].

2.4.2 Extreme Gradient Boosting (XGBoost)

XGBoost predicts a quantitative target with an additive ensemble of regression trees. A regression tree splits the feature space into rectangular regions by asking threshold questions of the form "is feature X_j below some value?", assigning a constant prediction to each region [21]. Because the splits are chosen from the data, a tree can represent curved relationships and interactions without either being specified in advance. Boosting combines many small trees rather than relying on one large one, fitting them sequentially so that each new tree addresses what the current ensemble still fails to explain; for a general loss the trees are fitted to its gradients [11].

The model consists of three components: K regression trees, each defined by its structure, which determines the leaf an observation falls into, and by a continuous score attached to each leaf; the additive rule that combines their outputs; and a regularisation term that charges for tree complexity within the fitted objective [12].

For an observation x_i the prediction is the sum of the outputs of all K trees,

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i),$$

where $\hat{y}_i$ is the predicted remaining life, K is the number of trees, and f_k is the k -th tree. Unlike a classification tree, each leaf here carries a numerical value, and the prediction is obtained by following the decision rules of every tree and summing the scores of the leaves reached [12].

The ensemble is learnt by minimising

$$L = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \quad \text{where} \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2,$$

in which l is a differentiable convex loss measuring the discrepancy between the prediction $\hat{y}_i$ and the observed target y_i , T is the number of leaves in a tree, w is the vector of leaf scores, γ charges a fixed cost for every additional leaf, and λ shrinks the leaf scores towards zero [12]. The second term therefore penalises complexity directly, favouring ensembles built from simple but predictive trees. At each iteration a tree is added greedily: candidate splits are evaluated using a second-order approximation of the loss, that is, using both its gradient and its curvature at the current prediction, and the one giving the greatest improvement is chosen [12].



Figure 2.1. Additive prediction of remaining useful life by two gradient-boosted regression trees. Each tree routes an observation to one leaf, and the prediction is the base score plus the score of the leaf reached in every tree. Created by the author based on [11] and [12].

Figure 2.1 shows how two trees combine into a single prediction. Non-linear relationships between sensor features and remaining life, and interactions among those features, are captured automatically without being named in advance. Because splits depend only on the ordering of feature values and not on their magnitudes, no rescaling of the features is required. And correlated sensor features do not produce the coefficient instability described in Section 2.4.1, since the model makes no attempt to assign a single weight to each feature.

Permutation importance measures how far a model's error rises when the values of one input, or of a group of inputs, are shuffled at random across observations while everything else is left

as recorded [21]. Writing E for the error of the fitted model on the data as recorded and $E_{p(j)}$ for its error after the values of input j have been shuffled, the importance of that input is

$$\text{Importance}(j) = E_{p(j)} - E.$$

A large value indicates that the fitted model was drawing on that input; a value near zero indicates that it was not, given what the remaining inputs supply. The measure is not additive, so the importance of a group of inputs cannot be obtained by summing the values of its members: where inputs are correlated, shuffling one alone allows the others to compensate, and a group must therefore be permuted jointly. The shuffling is also marginal rather than conditional, so it can produce combinations of values that do not occur in the recorded data. The measure describes one fitted model, not the underlying relationship, and it is applied in this thesis only as a post-hoc inspection (Section 7.5).

The method introduces configuration choices with no counterpart in linear regression, among them the number of trees, their maximum depth, and the learning rate controlling how much each successive tree contributes; these must be selected rather than derived. And shared signal within a correlated group of sensors may be attributed to one member instead of another, which affects which splits are chosen and how the importance of individual features appears.

In this thesis XGBoost is applied to the same summary features as linear regression (Section 6.4.2), so the difference between the two isolates the value of non-linearity and feature interactions on identical inputs. The configuration used and the procedure that selected it are given in Section 6.5.2. Full derivations are given in [12], and the underlying framework in [11].

2.4.3 Long Short-Term Memory (LSTM)

An LSTM is a recurrent network that processes a sequence one step at a time, carrying an internal state forward so that the representation of the current step depends on what preceded it. This lets the network derive temporal structure from the ordered measurements instead of receiving it pre-digested through summary features [13].

The model consists of a memory cell and three multiplicative gates. The cell state is a vector carried from one step to the next, and access to it is regulated by an input gate controlling what enters, a forget gate discarding what has ceased to be relevant [22], and an output gate controlling what is read from it. Each gate is a small learned layer whose output passes through a sigmoid, so that it acts as a continuously adjustable valve, closed at zero and fully open at one [18]. This three-gate cell is the form used in this thesis.

The mechanism that distinguishes the architecture is the way the cell state is updated at each step,

$$c_t = f_t \otimes c_{t-1} + i_t \otimes \widetilde{g}_t,$$

where c_t is the cell state at step t , c_{t-1} the state carried in from the previous step, f_t the forget-gate activation, i_t the input-gate activation, $\widetilde{g}_t$ the candidate values proposed at the current step, and $\otimes$ element-wise multiplication. The state is carried forward by addition rather than by repeated multiplication, which creates a direct route for information and gradients through time and mitigates the vanishing and exploding gradient problem that limits simple recurrent networks to short spans [13], [18]. The mitigation is not absolute: the contribution of an old

state to a much later one is scaled by the product of the intervening forget-gate activations, and that product can still become small if those gates remain closed over many steps. The gate equations themselves are given in [13] and [22].

Figure 2.2 shows the structure of one such cell. The LSTM is recurrent: it has no attention mechanism and processes a sequence strictly in order, with information about earlier steps reaching later ones only through the cell and hidden states. The model used in this thesis is the standard forget-gate LSTM, not a Transformer variant, and the two are treated as distinct families (Section 2.4.4).

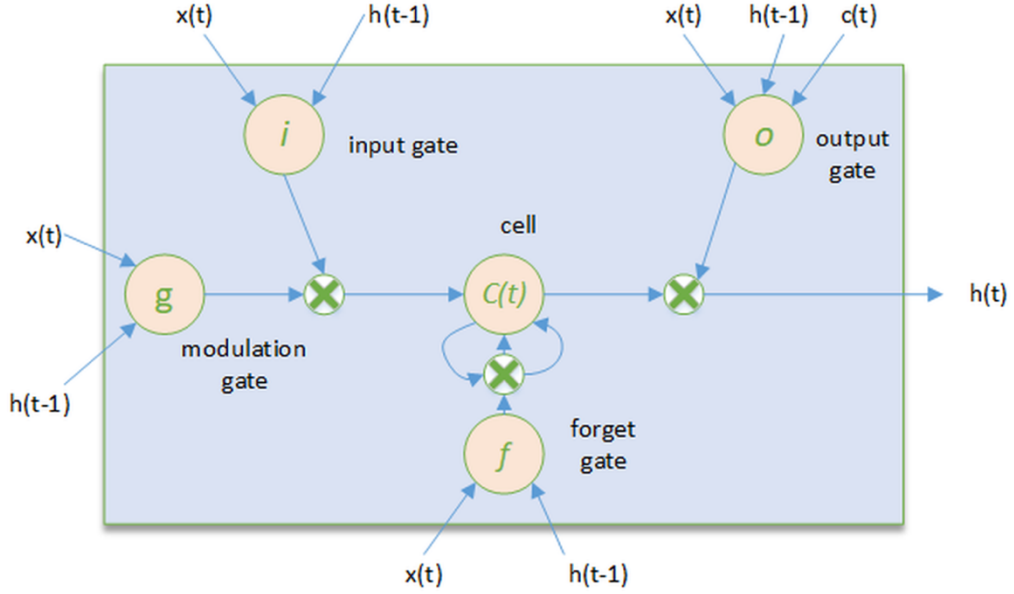


Figure 2.2. Structure of a standard forget-gate LSTM cell. The three gates regulate what enters the cell state, what is discarded from it, and what is read out; the shaded area encloses the cell, and the crossed circles denote element-wise multiplication [22].

The LSTM receives the ordered sensor measurements themselves, so temporal structure need not be specified in advance. Like any network with a large number of weights, an LSTM can fit the training data far more closely than it can generalise; the remedy applied here is dropout, which randomly removes units together with their connections during training so that the network cannot rely too heavily on any particular combination of them [14]. The number of weights also grows quickly with the hidden size. For a layer with input dimension d and hidden size H ,

$$N_{\text{LSTM}} = 4H(d + H + 1),$$

where the factor four reflects the four internal layers of the cell, each contributing a weight matrix acting on the input and the incoming hidden state together with a bias vector; the parameters of any subsequent dense layers are added to this.

The weights are adjusted iteratively, one pass over the training data being called an epoch, by an optimiser that uses the gradient of the loss; the optimiser used in this thesis is Adam, which adapts the step size of each weight individually from running averages of its recent gradients, and whose base step size is set by a learning rate [10]. Training for too many epochs reproduces the overfitting described in Section 2.2, so early stopping is applied: a portion of the data is held

out from fitting, the loss on it is monitored after each epoch, and training halts once that loss stops improving [10]. The held-out portion is therefore not available for fitting, which has a consequence for this comparison recorded in Section 6.2.1. In this thesis the LSTM receives the ordered sensor sequences; how they are padded and masked, and where dropout is applied, are specified in Section 6.4.3.

Sequences are presented to the network in batches, and those within one batch must share a common length. Where the recorded sequences differ in length, the shorter ones are extended with filler rows until they match the longest, a procedure known as padding. A masking layer can then mark the filler positions, so that the recurrent layer treats them as absent rather than as observations.

2.4.4 Chronos-2

Chronos-2 is a pretrained time-series model. Instead of training a separate model for every dataset, a single model is trained once on large-scale time-series data and then applied across different forecasting problems, which removes the need to train from scratch for each use case [17]. Such a model can, in principle, produce forecasts on data it has never seen during training, a capability referred to as zero-shot forecasting [17].

The model consists of three components. Input series are normalised by a robust scaling scheme and split into non-overlapping patches, each mapped by a residual network to a high-dimensional embedding, that is a vector of continuous numbers in which the network encodes the content of that patch. A stack of Transformer blocks then operates on these patch embeddings, each block alternating between a time attention layer, which aggregates across patches within one series, and a group attention layer, which aggregates across the series of a group at each patch index, where a group may be a set of related series or the variates of one multivariate series [17]. In the published pipeline a quantile head finally produces a probabilistic forecast; that head is not used here, for the reason given below.

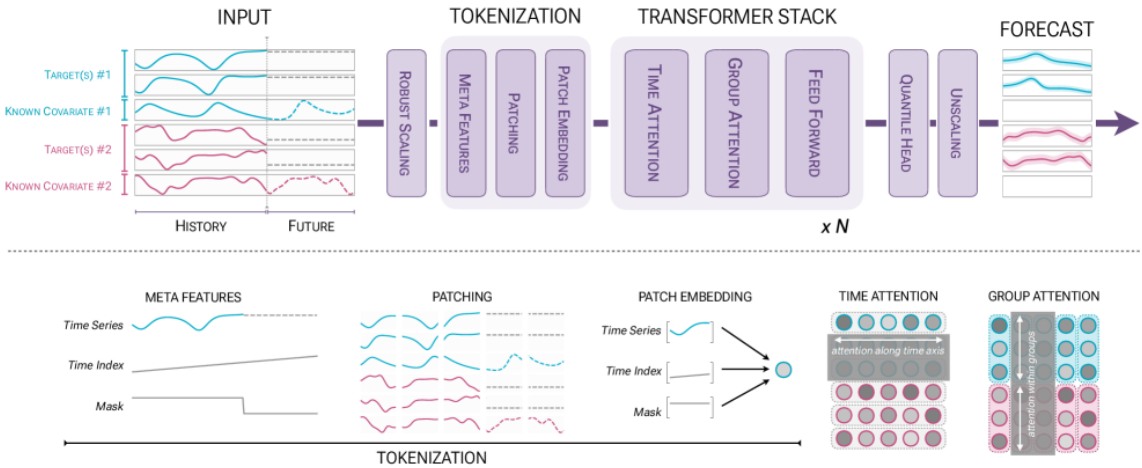


Figure 2.3. The Chronos-2 inference pipeline. Input series are robustly scaled, split into patches, and mapped to embeddings; the Transformer stack alternates between time attention and group attention before a quantile head produces the forecast. In this thesis the pipeline is used only up to the internal representations, and the quantile head is not applied. Reproduced from Ansari et al. [17].

Figure 2.3 shows how the three components fit together. The architecture underlying the Transformer blocks is attention, which relates all positions in a sequence to one another directly,

allowing any position to draw on any other in a single operation regardless of the distance between them [15]. Each position is represented by a query, a key and a value vector, and the similarity between a query and every key determines how much of each value that position receives,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V,$$

where Q, K and V collect the query, key and value vectors of all positions and d_k is the dimension of a key; dividing by $\sqrt{d_k}$ keeps the similarities in a range where the softmax remains sensitive [15]. Because attention here operates on patch tokens instead of individual time steps, its cost grows with the square of the number of patches rather than the number of cycles.

The model handles univariate, multivariate, and covariate-informed tasks within one framework through the group attention mechanism [17], which is directly relevant here since the turbofan data are multivariate, with several sensors recorded simultaneously for each engine.

Chronos-2 has the largest stored backbone of the four models and the highest representation-extraction cost, since every sequence for which a prediction is required must be passed through the pretrained network; freezing the backbone removes the cost of backpropagating through it but not the cost of these forward passes, which dominate the computational effort associated with this model. And because the backbone is frozen, the number of parameters actually fitted to the turbofan data is smaller than that of the LSTM. Stored model size, task-specific trainable parameters, and measured computation time are therefore treated as three separate dimensions of complexity rather than as a single ordering.

The backbone is neither retrained nor fine-tuned: it is used in the state released by its authors, with its weights held fixed and excluded from the optimiser, so no property of the turbofan data enters them [17]. It is used not as a forecaster but as a frozen feature extractor: the process stops at the internal representations produced by the Transformer stack, which are taken as fixed numerical descriptions of the input sequences, and only an external regression head fitted on top of them is trained on the turbofan data. The system as a whole is therefore not zero-shot with respect to remaining useful life, even though the backbone that produces the representations was never exposed to this task. The sequence representation supplied to it, and the pooling and head configuration, are specified in Section 6.4.3.

Chronos-2 is included to test a specific proposition. The three preceding models learn everything they know from the turbofan data; Chronos-2 arrives with a representation of temporal behaviour already formed from unrelated data, and the question is whether that transfers to a specialised engineering task it was never trained on. Its comparison with the LSTM is the informative one: both operate on ordered sensor sequences, so where their channels and horizons are matched, the comparison reflects architecture and pretraining together. The full architecture and pretraining procedure are described in [17], the original Chronos formulation in [16], and the Transformer in [15].

2.5 Evaluation Metrics

Different metrics respond to different aspects of the error distribution, so three are used here: two standard regression measures and one specific to prognostics. Following the C-MAPSS protocol, one prediction is produced for each test engine at its final observed cycle and compared with the true remaining life supplied for it [3]. In all three definitions below, N denotes the number of test engines rather than recorded cycles, $\hat{y}_i$ the predicted remaining life of engine i , and y_i its true value.

The mean absolute error is the average magnitude of the prediction errors,

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|.$$

Expressed in the same unit as the target, it states directly how far a typical prediction lies from the truth in flight cycles, with each error contributing in proportion to its magnitude.

The root mean squared error instead averages the squared errors before taking the square root,

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}.$$

Squaring gives greater weight to large deviations, so RMSE responds more strongly than MAE to a few badly predicted engines; where it substantially exceeds MAE, some predictions carry considerably larger errors than the rest.

Both are symmetric criteria, weighting predictions that fall short of the true value and those that exceed it equally [6], and that symmetry is what makes them incomplete for maintenance applications. Saxena et al. observe that in prognostics the key aspect is to avoid failures, so predicting early is generally preferable to predicting late, while noting that the preference is not universal: where failure is not life-threatening and early predictions carry an economic burden, a performance measure should reflect the requirements of the specific application [3]. This thesis therefore reports an asymmetric criterion alongside the two symmetric ones, for the reason given below and because it preserves comparability with previous C-MAPSS studies.

The third metric comes from outside the standard regression toolkit. When NASA released the C-MAPSS trajectories for the 2008 Prognostics and Health Management (PHM) Data Challenge, the organisers needed one number by which to rank the competing entries, and neither of the two metrics above expressed what the task was about: in maintenance the cost of an error depends on its direction, not only on its size. Saxena et al. therefore defined a scoring function that penalises a prediction exceeding the true remaining life more heavily than one falling short of it by the same amount [3]. It is referred to throughout this thesis as the PHM score and appears in parts of the literature as the NASA score. The reason for the asymmetry is operational rather than statistical: an engine predicted to last longer than it does may be left in service past the point at which maintenance should have been scheduled, so the consequence is an unplanned failure, whereas an engine predicted to fail early costs an inspection that was not

yet necessary. A perfect set of predictions scores zero; lower values are better. Figure 2.4 shows the shape of this penalty.

The penalty is exponential rather than quadratic, and the two directions use different rates, so the curve is not symmetric about zero. The constants were fixed by the challenge organisers for this dataset rather than derived from the maintenance costs of a particular fleet, so the score states the direction and the steepness of the asymmetry rather than a monetary quantity. It has been reported alongside the root mean squared error in most subsequent C-MAPSS work, which is the second reason for using it here: it keeps the results of this thesis comparable with the published figures of Chapter 3.

Writing d_i for the difference between the estimated and the true remaining life of engine i , so that d_i is negative for an early prediction and positive for a late one, the score is defined piecewise:

$$S = \sum_{i=1}^N s_i, \quad \text{where} \quad s_i = \begin{cases} e^{-d_i/13} - 1, & d_i < 0 \\ e^{d_i/10} - 1, & d_i \geq 0 \end{cases}$$

with divisors of 13 cycles for an early prediction and 10 for a late one. Saxena et al. state the two constants as 10 and 13 [3]; they are assigned here so that the late branch carries the smaller of the two, which is what produces the asymmetry the challenge organisers describe and is the form used throughout the subsequent C-MAPSS literature. The penalty grows exponentially with the magnitude of the error in both directions, so a moderate error costs little while a large one costs a great deal. An error of twenty cycles contributes approximately 3.7 if early and 6.4 if late.

The score is not expressed in an interpretable unit, so its absolute value is useful only for comparison; it is a sum rather than an average, so it grows with the number of test engines and cannot be compared across test sets of different sizes; and because the penalty is exponential, a single badly predicted engine can dominate the total, so the score reflects worst-case behaviour more than typical behaviour. All three metrics are reported together, and they need not agree: a model that tends to predict late may achieve a competitive MAE while scoring poorly. Where they disagree, the disagreement is reported rather than resolved by choosing a single figure of merit, and what it indicates is examined in Section 7.2.

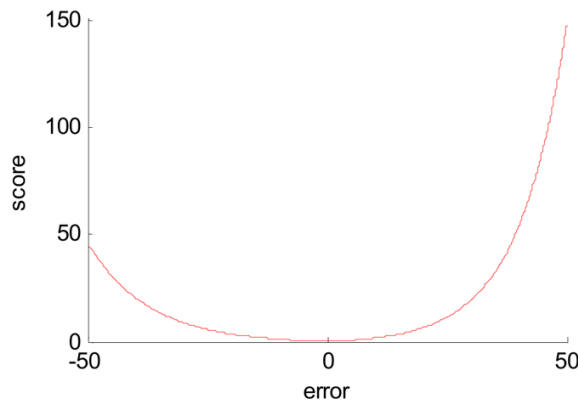


Figure 2.4. The PHM score as a function of prediction error. Negative errors correspond to early predictions and positive errors to late ones; the curve is steeper on the positive side, so a late prediction is penalised more heavily than an early one of equal size. Reproduced from Saxena et al. [3].

2.6 Statistical Comparison

Comparing two models on any of these metrics raises a further question: whether an observed difference is larger than the variation that would arise anyway. Because each engine is evaluated by both models, the errors form matched pairs, and a paired test is appropriate. The Wilcoxon signed-rank test compares two sets of paired measurements by ranking the differences between them and asking whether those differences are distributed evenly around zero; unlike the corresponding parametric test it does not assume that the errors follow a normal distribution [29].

Such a test returns a p-value: the probability of observing a difference at least as large as the one measured if the two models in fact performed equally. A small p-value therefore counts as evidence against that assumption, and a threshold fixed in advance, five per cent throughout this thesis, decides which differences are reported as significant. A p-value above the threshold is an absence of evidence for a difference, not evidence that none exists.

Where several such comparisons are made on the same data, the chance that at least one appears significant by accident rises with their number. Holm's step-down procedure corrects for this: the p-values are ordered from smallest to largest and each is assessed against a threshold that becomes less strict down the list, so that the probability of at least one false rejection across the whole family remains controlled [28]. A simpler correction, the Bonferroni method, divides the significance level by the number of tests and applies the same stricter threshold to all of them; it is the more conservative of the two [29]. Where two groups of observations are independent rather than paired, the corresponding non-parametric comparison is the Mann-Whitney U test, which asks whether the values in one group tend to be larger than those in the other without assuming any particular distribution [29]. These tests are applied in Sections 5.6 and 6.5.3.

3 Related Work

3.1 Scope of the Review

This chapter reviews previous work on RUL prediction using the C-MAPSS benchmark, with attention directed at FD001. The restriction reflects a structural difference between the subsets, described in Section 4.3: they differ in operating conditions and fault modes, and in more than one respect at a time. Reported performance follows that structure, and Guilherme found error to be noticeably lower on FD001 and higher on FD002, attributing the difference to the variability introduced by multiple operating regimes [20]. Comparing a figure obtained on FD001 with one obtained on FD004 accordingly compares two different problems. Only FD001 results are therefore included in the direct numerical comparison, although studies using other subsets are discussed where they offer relevant methodological insight.

Most published work on this dataset reports RMSE and a substantial part of it the PHM score, but MAE far less often, so the published figures permit comparison on some of the three criteria used here and not on others. Studies also differ in whether the target is linear or piecewise-linear and, where it is capped, in the value chosen; as Section 2.3 set out, this determines what the model is asked to learn, so two studies using the same architecture, but different target definitions are not evaluating the same task. They differ finally in experimental protocol, in the length of the input window, the preprocessing applied to the sensor signals, and the number of sensors retained, all of which affect the result independently of the model. Published figures are accordingly treated here as indicative of the range achieved on this subset rather than as directly commensurable, and the comparison table in Section 3.6 records for each study whether its setup is comparable to the one adopted in this thesis.

The review is organised by model family rather than chronologically, since the question addressed by this thesis concerns families of models rather than the order in which they appeared. The architectures themselves were described in Section 2.4 and are not re-explained; what follows concerns how they have been applied to this dataset and what has been reported.

3.2 Recurrent and Convolutional Approaches

The earliest influential contribution is that of Heimes, who applied a recurrent neural network to the PHM 2008 Data Challenge and, in the same work, introduced the piecewise target discussed in Section 2.3 [4]. The challenge dataset he used is not FD001 but a larger fleet whose shortest run lasts 127 cycles, so his reported figures are not directly comparable with the ones that follow. His work is treated here as the origin of the capped target rather than as a comparable FD001 result.

Subsequent work applied deeper networks of two kinds. Zheng et al. used stacked LSTM layers to model the sensor sequences directly [8]. Li et al. instead applied a deep convolutional network, in which filters extract degradation features from a window of sensor readings without any recurrent connection [9]. Ellefsen et al. combined an unsupervised pretraining stage with an LSTM network [27]. All three reach comparable errors on this subset, collected with the rest in Table 3.1, though for the reasons given in Section 3.1 the small numerical gap between them cannot be attributed to architecture alone.

Asif et al. placed greater emphasis on preprocessing and target construction alongside the LSTM architecture itself [6]. Their pipeline selects sensors that exhibit strong correlation and monotonic degradation trends with respect to remaining life; filters the retained signals with a moving median; and replaces the fixed cap with an improved piecewise degradation model that locates the point at which deterioration begins. Their reported error distributions concentrate in a narrow band on FD001 and spread more widely on FD002 and FD004, which they attribute to the irregular behaviour introduced by six operating conditions [6]. This work is relevant here because it demonstrates that decisions about preprocessing and labelling can be examined systematically rather than fixed by convention.

3.3 Transformer-Based Approaches

Attention-based models entered this field on an explicit argument about the limitations of both preceding families. Mo et al. adopted the Transformer encoder as the backbone of their model, observing that a standard convolutional layer accesses only a local neighbourhood determined by its kernel size, whereas self-attention directly relates all positions within the input window, and that unlike recurrent methods it can fully exploit parallel computation [23]. In the same work they identified a weakness of the architecture and corrected it: because attention renders the resulting high-level features insensitive to local context, they added a gated convolutional unit to restore the model's ability to incorporate local information at each time step, giving the GCU-Transformer [23].

Later variants have followed the same pattern of augmenting the base architecture rather than relying on attention alone. Li et al. proposed DSFormer, an encoder–decoder model combining attention along both the sensor and the time dimensions with a temporal convolutional network and a decoder-side decomposition module supplying trend information [24]. Ma et al. combined a Transformer encoder with an adaptive graph convolutional network, arguing that fully connected layers along the sensor dimension are constrained only by the loss function, so the weights they learn need not reflect the true relationships between sensors, whereas a graph structure imposes those relationships explicitly [25]. Their own comparative table lists the GCU-Transformer at 22.81, roughly twice the error its authors reported; since the two figures originate from different studies, gaps of this kind should be interpreted cautiously [25].

These studies share a design choice: self-attention models the global temporal dependencies, while convolutional, graph-based or decomposition modules supply local patterns and inter-sensor structure. The reported gains therefore belong to the combined architecture rather than to attention alone. Ma et al.'s ablation supports this: removing the graph component raised the average RMSE from 13.74 to 14.26, while removing the Transformer raised it to 21.19 [25].

One controlled comparison did not favour the Transformer on FD001. Guilherme trained an LSTM and an encoder-only Transformer under the same experimental protocol and reported that the LSTM achieved the lower error on FD001, with the same ordering on FD003; the Transformer was superior only on FD002, the multi-condition subset [20]. He attributes this to the nature of the subsets, suggesting that the Transformer captured the variability of multiple operating regimes more effectively, and concludes that a baseline encoder-only Transformer can be competitive but does not reach the level of the hybrid architectures discussed above [20]. This is useful evidence that a newer architecture does not automatically outperform an LSTM

on FD001. It is not, however, a direct precedent for the pretrained model examined in this thesis, because Guilherme's Transformer was trained from scratch, whereas Chronos-2 is used here as a frozen pretrained feature extractor (Section 2.4.4).

3.4 Simple and Interpretable Models

The models reviewed so far are all deep architectures. Tabular and linear methods occupy a much smaller place in this literature, and where they appear it is usually as a baseline against which a deeper model is measured rather than as an object of study in their own right. Felício states the position directly: although XGBoost produces results comparable to random forests and other established machine-learning techniques in prognostics, it has seen less presence in the field than those alternatives [18].

Where simple models have been studied deliberately, the motivation has generally been transparency rather than accuracy. Yıldırım and Afşer proposed a linear classification–regression framework in which linear discriminant analysis establishes hyperplane boundaries partitioning the measurement space into nominal, warning, and failure regions, and the signed distance from each observation to those boundaries is used as the feature from which remaining life is predicted [26]. Their own assessment is explicit: validation on C-MAPSS showed that the framework does not achieve state-of-the-art performance, but offers significant advantages in interpretability, computational efficiency, and ease of deployment, with the learned hyperplane coefficients directly revealing which sensor measurements most strongly indicate degradation [26]. The resulting errors, given in Table 3.1, make the accuracy–interpretability trade-off concrete, although the training procedure pooled units from all four C-MAPSS subsets and therefore differs from an FD001-only protocol.

In Felício's initial benchmark, conducted before the BiLSTM was optimised, random forest and XGBoost achieved lower RMSE values than the BiLSTM while the BiLSTM achieved a lower MAE. The comparison precedes optimisation of the deep model, and the work is evaluated on N-CMAPSS rather than on the subset used here, so the figures are not comparable with the remaining entries. A further observation from the same work concerns how the input is constructed. With the model held fixed, Felício reported that selecting features by Pearson correlation degraded performance relative to using all available features, whereas selecting them by the importance ranking produced by a random forest improved it substantially [18]. The experiment used one subset and one architecture, so it does not establish that the same ranking of selection methods would hold elsewhere; it does establish that the choice of which features reach the model can affect the result as much as choices about the model itself.

3.5 The Role of Target Definition and Input Representation

The studies above suggest that decisions taken before the model sees the data can matter as much as the model itself. Two have been tested directly: how the target is defined, and how much operating history the model receives.

Guilherme's controlled comparison shows how large the first effect is. On FD001 his LSTM runs gave MAE values between 20.57 and 22.42 under the linear target and between 9.14 and 11.33 under the piecewise one, with his Transformer runs behaving the same way [20]. He concludes that the RUL formulation is the single largest factor in this setting, and that stabilising

the target function is a prerequisite for training deep models effectively on this dataset [20]. The ranking of the two architectures did not move, the LSTM staying ahead under both; what moved, by a wide margin, was the performance that could be reached.

The cap itself is a convention rather than a finding. Most of the work reviewed here takes the value of 125 from earlier studies instead of re-estimating it: Ma et al. adopt it on that basis and on the observation that degradation is negligible early in an engine's life, so the target can be held constant there [25]. Asif et al. instead locate the onset of deterioration for each engine separately [6]. The cap is a simplification open to refinement, not a property of the data.

The evidence on input length is thinner than it looks. Guilherme compared windows of 50 and 60 cycles and found only small differences on FD001 once the piecewise target was in use [20]. Ten extra cycles added little under a single operating condition, and the experiment says nothing about a substantially longer history. On FD002 the same increase helped clearly only alongside a more selective feature set and the piecewise target [20], which points to an interaction between input length, feature selection and subset complexity rather than an effect of length alone.

All of this sits inside sequence models, where the input is always an ordered window and only its length changes. The question addressed is how much history is supplied, not the form it takes.

3.6 Positioning of the Present Work

Table 3.1 collects the results discussed in this chapter and records, for each study, how closely its experimental setup aligns with the protocol adopted in this thesis. The figures for Guilherme are those obtained under the piecewise target; the full ranges are given in Section 3.5. The last column states whether a study's protocol is broadly aligned with the one used here. The studies marked as aligned were evaluated on the same 100 FD001 test engines, while Yıldırım and Afşer pooled all four subsets and Felício evaluated on N-CMAPSS, so those two entries are not comparable with the rest. Even the aligned studies differ in how the target is defined (Section 3.1), so the table locates the present results within the published range rather than ranking the studies against one another.

Table 3.1. Selected results relevant to the FD001 comparison and alignment with the present protocol.

Study	Model family	MAE	RMSE	PHM score	Broad alignment? protocol
Zheng et al. [8]	Recurrent (LSTM)	—	16.14	338	Broadly aligned
Li et al. [9]	Convolutional (DCNN)	—	12.61	273.7	Broadly aligned
Ellefsen et al. [27]	Semi-supervised + LSTM	—	12.56	231	Broadly aligned
Mo et al. [23]	Transformer + gated convolution	—	11.27	—	Broadly aligned
Li et al. [24]	Transformer (DSFormer)	—	10.77	199.82	Broadly aligned
Ma et al. [25]	Transformer + graph convolution	—	12.46	—	Broadly aligned
Guilherme [20]	Recurrent (LSTM)	9.14	12.57	—	Broadly aligned

Study	Model family	MAE	RMSE	PHM score	Broad alignment? protocol
Guilherme [20]	Transformer (encoder-only)	10.34	14.35	—	Broadly aligned
Yıldırım & Afşer [26]	Linear (LDA + regression)	—	30.40	$\approx 4.7 \times 10^3$	No, training pooled across all four subsets
Felício [18]	Random forest / XGBoost / BiLSTM	—	—	—	No, evaluated on N-CMAPSS

Lower values are better for all three metrics. The PHM score is a sum over the 100 test engines, so it is not expressed in cycles.

Broad alignment means training and evaluation on FD001 alone, a piecewise target, and one prediction at each test engine's final observed cycle. It does not mean identical caps, sensor sets, window lengths or selection procedures: Zheng et al. use a cap of 130 [8], Ellefsen et al. tune it between 115 and 140 [27].

Every aligned entry is a deep architecture, and the two entries with simple or tabular models are the two that are not: one pooled the four subsets in training, the other used a different dataset. The reviewed work therefore holds no comparable FD001 result for a linear or tree-based model. That is the first gap.

All eight aligned entries report RMSE, four the PHM score, and only Guilherme reports MAE [20], the size of a typical error (Section 2.5). Training time and model size are too inconsistent to compare. Accuracy is thus better documented than typical error or cost. That is the second gap; this thesis reports all three metrics together with the cost.

The studies that vary the target or the input length stay inside sequence models, where the input is always an ordered window. None gives the same history to different model families under one protocol, so a reported gap between a tabular and a sequence model cannot be separated from the fact that each received the history in a different form. That is the third gap, and the present design follows from it.

Sections 1.3 and 1.4 address all three. The aim is not to beat the best published figure, but to show what a difference between model families can be attributed to once the conditions are stated.

4 The C-MAPSS Dataset

4.1 The C-MAPSS Simulator and the Turbofan Engine

The data are not measurements from physical engines. They are generated by the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS), developed at the NASA Ames Research Center, which models a large commercial turbofan and produces sensor readings as it operates and deteriorates [3]. Running real engines to failure would be prohibitively expensive, whereas a validated simulator can produce complete run-to-failure histories for many engines under controlled conditions [3]. The training data therefore describe the full life of each simulated engine, from a healthy initial state to failure; the test trajectories stop earlier, as Section 4.5 explains. Figure 4.1 shows the main components of the turbofan engine and their arrangement along the gas path.

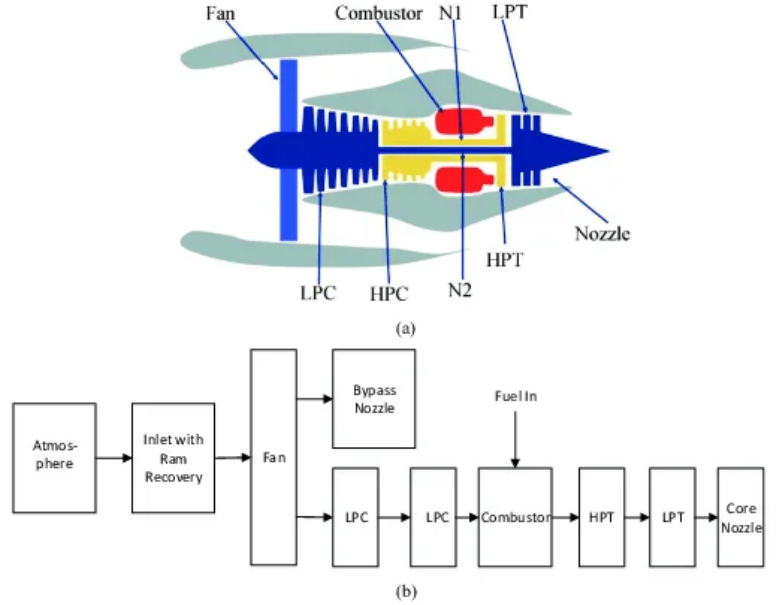


Figure 4.1. Simplified diagram of the turbofan engine modelled by the C-MAPSS simulator. The components are arranged along the gas path: fan, low-pressure compressor (LPC), high-pressure compressor (HPC), combustor, high-pressure turbine (HPT), low-pressure turbine (LPT), and nozzle. The labels N1 and N2 denote the low-pressure and high-pressure spool speeds. Adapted from [3].

Understanding what the sensors measure requires knowing the basic structure of the engine. A turbofan produces thrust by drawing in air, compressing it, mixing it with fuel and burning it, and expelling the resulting high-energy gas. This takes place across a sequence of components arranged along the gas path, each monitored by the C-MAPSS sensors. Air enters through the fan, the large rotating blade assembly at the front, where part of it passes into the engine core while the remainder bypasses the core and contributes directly to thrust. The air entering the core is raised in pressure by the low-pressure compressor (LPC) and compressed further by the high-pressure compressor (HPC) to the level required for combustion. In the combustor, fuel is added to the compressed air and ignited, producing high-temperature, high-pressure gas. The

high-pressure turbine (HPT) extracts energy from that gas to drive the HPC, and the low-pressure turbine (LPT) extracts further energy to drive the fan and the LPC. The remaining gas is expelled through the nozzle, producing thrust.

The engine is built around two rotating shafts, or spools. The high-pressure spool connects the HPC and the HPT, while the low-pressure spool connects the fan, the LPC, and the LPT. The two rotate at different speeds, and those speeds are among the quantities measured by the sensors listed in Section 4.4.

Each simulated engine begins in a healthy state and develops a fault at some point during operation. The fault grows over successive cycles until the engine can no longer perform its function, at which point it is considered to have failed [3]. Degradation alters the engine's internal behaviour, and that change is reflected in the sensor measurements. The premise of the dataset, and of this thesis, is that the relationship can be exploited in reverse: by observing how the measurements change, it is possible to estimate how far an engine has progressed towards failure.

4.2 Dataset Structure and Feature Categories

The FD001 data are provided as plain text files in which each row corresponds to one operating cycle of one engine and each column to one recorded variable. Every row is therefore a single snapshot of one engine at one point in its life, and the rows belonging to the same engine form the complete history of that engine. Each row contains 26 columns, falling into the three categories summarised in Table 4.1.

Table 4.1. The three categories of columns in the FD001 dataset.

Category	Columns	Count	Role
Identifiers	unit, cycle	2	Specify which engine the row belongs to and its position in time
Operational settings	op1, op2, op3	3	Describe the flight condition under which the cycle was recorded
Sensor measurements	s1–s21	21	Physical measurements of the engine's internal state

The identifiers organise the data rather than describe the engine's state; the cycle column provides the time axis along which degradation develops. The operational settings affect the engine's behaviour and therefore its readings [3], and Section 4.3 explains why the three are effectively constant throughout FD001. The sensor columns are the core of the dataset, each recording a physical quantity measured at a point in the gas path.

Two consequences of this layout govern everything that follows. A row is not an engine: one engine occupies as many consecutive rows as it has recorded cycles, so the 20,631 rows of the training file describe 100 engines rather than 20,631 independent cases, and the number of rows per engine differs because engines fail at different ages. And because the sensor columns change from row to row within an engine while the identifiers only locate the row in time, each engine is a multivariate time series rather than a fixed set of measurements. Table 4.2 shows the opening cycles of two training engines.

Table 4.2. Opening cycles of two training engines, showing four of the twenty-one sensor columns. Engine 1 occupies 192 consecutive rows and engine 2 a different number; the cycle column restarts at one for each engine.

unit	cycle	s2	s3	s4	s11
1	1	641.82	1589.70	1400.60	47.47
1	2	642.15	1591.82	1403.14	47.49
1	3	642.35	1587.99	1404.20	47.27
2	1	641.89	1583.84	1391.28	46.93
2	2	641.82	1587.05	1393.13	47.24
2	3	641.55	1588.32	1398.96	47.22

The two engines already differ at their first recorded cycle. In s11 the gap between them is 0.54 psia, against a mean change of 0.796 over a complete engine life (Section 5.4.3). An engine's current reading therefore does not by itself locate it within its degradation.

Where a record ends differs between the two files, and the distinction governs the whole experiment. A training engine is recorded up to the cycle at which it fails, so its remaining life is known at every cycle of its history and every cycle can serve as a training example. A test engine is recorded up to an earlier cycle chosen by the data providers, so its history stops while the engine is still operating; its remaining life at that final cycle is not recoverable from the file and is supplied separately in a ground-truth file. Section 4.5 gives the construction of the labels in both cases.

4.3 Operating Condition and Fault Mode

The complete C-MAPSS dataset is divided into four subsets, FD001 to FD004. They are generated by the same simulator and describe the same type of engine but differ in two respects: the number of operating conditions under which the engines are run, and the number of fault modes through which they degrade [3]. Together they determine how difficult each subset is to model. FD001, used in this thesis, is the simplest of the four.

Operating condition. An operating condition is the external flight situation under which an engine is run, defined by parameters such as altitude, Mach number, and throttle-resolver angle [3]. The same engine in the same state of health produces different readings under different conditions, so where a dataset contains several of them the readings reflect two influences at once: the changing flight condition and the underlying degradation. FD001 contains only one [3]. All engines are simulated under the same flight situation throughout their lives, so changes in the readings are not confounded by changes in operating regime. This is why the three operational settings are effectively constant, as Chapter 5 shows.

Fault mode. A fault mode is the physical mechanism by which an engine degrades [3]. Degradation can occur in different components and in different ways, for example as a loss of efficiency in the high-pressure compressor or a reduction in flow capacity in the low-pressure turbine, and each affects the readings differently; where several are present, a model must implicitly tell them apart. FD001 contains one: degradation of the high-pressure compressor

[3]. The fault is introduced by progressively modifying the flow and efficiency of the HPC module from healthy values towards values corresponding to failure; no degradation is injected into the other modules, although their measured states may respond through the coupled engine and control system [3]. Every engine in FD001 therefore degrades through the same mechanism, differing in initial condition and in the rate at which the fault develops.

The other three subsets differ from FD001 by introducing multiple operating conditions (FD002), an additional fault mode (FD003), or both (FD004). A single operating condition combined with a single fault mode makes FD001 a controlled and comparatively simple prediction problem, in which the relationship between the sensor readings and the remaining life is not obscured by changing flight conditions or by competing failure mechanisms. That property is the basis of the scope decision stated in Section 1.4.

4.4 Sensors and Operational Settings

Figure 4.2 gives a first view of the raw records. It shows four sensors against the operating cycle for three training engines: the shortest-lived, one close to the median, and the longest-lived. The records are of unequal length, the longest engine running nearly three times as many cycles as the shortest. Each sensor moves systematically across a trajectory rather than fluctuating about a fixed level, although the movement carries cycle-to-cycle variation comparable in size to the change over tens of cycles. The direction differs between sensors, s11 rising and s12 falling as failure approaches, and the pace is uneven, s4 changing slowly over most of a trajectory and rapidly towards its end. Sensor s9 behaves unlike the other three: across these same engines it rises steeply in one, falls in another, and stays near its starting level in the third.

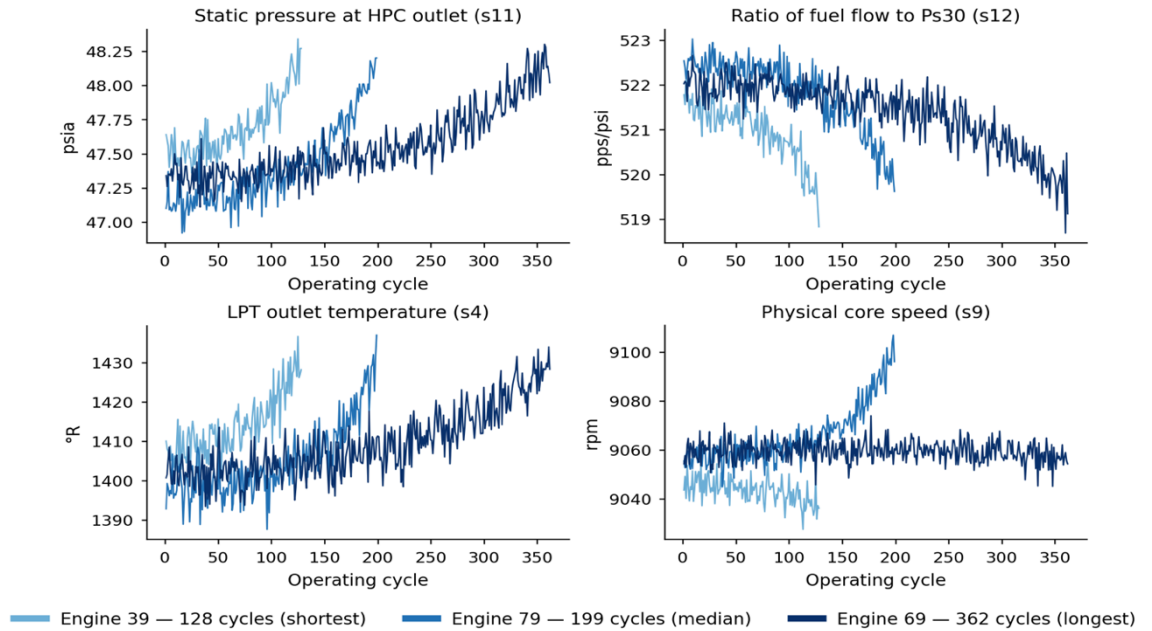


Figure 4.2. Recorded values of four sensors against operating cycle for three training engines of differing lifetime. Values are shown in their recorded units and without smoothing.

The sensors are numbered s1 to s21 in the raw data, and the simulator documentation additionally assigns each a symbolic name such as T_{24} or N_c [3]. Table A.1 in Appendix A gives both notations, together with the physical meaning, engine module, and unit of every variable,

so that the thesis can be related to the raw files and to the documentation alike. The symbolic names follow a convention: a leading T denotes a temperature, P a total pressure, Ps a static pressure, N a rotational speed, and W a flow rate. Units are degrees Rankine ($^{\circ}\text{R}$) for temperature, pounds per square inch absolute (psia) for pressure, revolutions per minute (rpm) for rotational speed, pounds per second (pps) for fuel flow and pounds-mass per second (lbm/s) for coolant flow [3]; the ratio of fuel flow to static pressure carries the compound unit pps/psi. Dimensionless quantities carry no unit.

Table A.1 describes what each variable represents. How these variables actually behave in FD001, how much each one varies, and which of them carry usable information about degradation, is examined in Chapter 5.

4.5 Remaining Useful Life Labels

Training a model to predict Remaining Useful Life requires associating each row with a target value: the number of operating cycles remaining for that engine at that cycle. The two definitions of this target, and the reasoning behind them, are set out in Section 2.3; their construction from the FD001 files is described below.

Target for the training engines. Each training engine is recorded until the cycle at which it fails, so its last recorded cycle is its failure cycle. The Remaining Useful Life at any earlier cycle is the number of cycles between that cycle and the failure cycle. This is the linear target of Equation (2.1), applied separately to each engine, since each has its own failure cycle, and it decreases by one for each operating cycle until it reaches zero at failure.

Target for the test engines. The test engines are not recorded until failure; their data stop at an earlier cycle. The true Remaining Useful Life at that final observed cycle therefore cannot be reconstructed from the data and is instead provided in a separate ground-truth file supplied with the dataset [3]. This file contains one value per test engine, giving the true number of cycles remaining at the engine's last recorded cycle. These values are used only for evaluation and never for training; the evaluation procedure that uses them is described in Section 6.5.3.

The two label variants. Both definitions are applied: the linear label of Equation (2.1) and the piecewise label of Equation (2.2). The value chosen for $R_{\max}$ and its justification are given in Section 6.3. Both variants are used in the comparison of SQ4 (Section 1.3), whose scope is stated in Section 7.4.

5 Exploratory Data Analysis

All analyses in this chapter, and all modelling decisions drawn from them, use the training data alone. The test input file is examined only to confirm that its structure, columns and data types match the training file; the ground-truth values are used only in the evaluation of Section 6.5.3. The chapter first shows what the recorded signals look like, then establishes the properties of FD001 that shape the target definition, the features, and the representation of each engine's history.

5.1 Analytical Scope and Data Integrity

This section establishes the size of the distributed files and whether their structure is sound; whether every column carries usable predictive information is examined in Section 5.3. The training file comprises 20,631 rows and the test input file 13,096, each covering 100 engines. The difference follows from the recording asymmetry described in Section 4.5, training trajectories averaging 206.3 cycles per engine against 131.0 for the test engines.

Four integrity checks were applied to both files, each addressing a condition that would change how the later analysis must be carried out. Missing values would call for imputation, and an imputed sensor reading inside a degradation trajectory is a value the engine never produced. Fully duplicated rows would give some cycles more weight than others in the row-level statistics of Section 5.7. Duplicated (unit, cycle) pairs would mean that a cycle index does not identify a single position in a trajectory, which the history-based features of Section 6.4.2 assume it does. Non-numeric entries would force a type conversion whose rule would have to be stated and defended.

The results are reported in Table B.1. All four checks returned zero for both files, which also confirms that the (unit, cycle) pair identifies each observation uniquely. No observations are removed or imputed during cleaning, and the preprocessing of Chapter 6 retains all 20,631 training observations.

5.2 Engine Lifetime Distribution

For each training engine, lifetime is its final recorded cycle, observable for all 100 because the trajectories extend to failure (Section 4.5). Figure 5.1 gives the distribution. Lifetimes range from 128 to 362 cycles, with a mean of 206.3 and a median of 199. The standard deviation is 46.3 cycles and the interquartile range extends from 177 to 229.2, so the middle half of engines fail within that interval. The mean exceeds the median by 7.3 cycles, indicating a mild right-skew, and the longest lifetime is 2.83 times the shortest.

Failure occurs at very different cycle indices, but this is not evidence that engines degrade at different rates: as Section 1.2 noted, engines begin with differing initial wear and a fault develops at some point in each trajectory, so a difference in lifetime may reflect the initial state, the onset, the later rate, or a combination. What it does show is that the cycle index cannot locate an engine within its degradation. At cycle 128 the shortest-lived engine has already failed, while the longest-lived still has 234 cycles remaining. Section 5.4.3 examines whether the sensor measurements vary between engines in a comparable way.

The constant used to cap the target is bounded in the same way. The shortest engine has a maximum linear target of 127, so a cap at or above that value leaves its labels unchanged, while any cap below it alters at least one label in every trajectory. This bound is a property of the FD001 training distribution rather than a general requirement, and it does not determine an optimal value; the choice is made in Section 6.3.

Sample weighting is affected as well. When each recorded cycle is one training sample and all rows carry equal weight, engines contribute in proportion to their lifetime: the longest supplies 362 samples against 128 from the shortest. Row-level statistics and row-level training objectives therefore give greater weight to longer trajectories. No engine-level weighting is applied in this work.

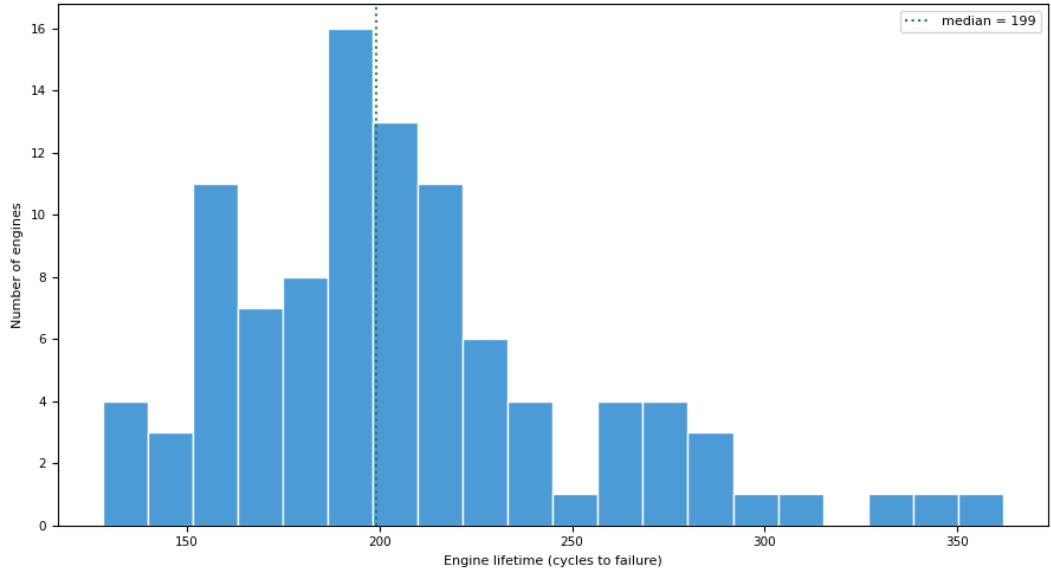


Figure 5.1. Distribution of observed lifetimes across the 100 FD001 training engines. The dotted green line marks the median lifetime of 199 cycles.

5.3 Variable Screening and Redundancy

Sections 4.2 and 4.3 indicated that the three operational settings should be effectively invariant within FD001. This section verifies that expectation, applies the same examination to the twenty-one sensors, and then tests the survivors for redundancy.

The standard deviation, the number of distinct recorded values, and the observed range were computed for each of the twenty-four candidate variables across all 20,631 training observations. Because the variables carry different units (Section 4.4), the standard deviations are not compared across variables.

The screening produced the three groups summarised in Table 5.1, and ten variables are removed on that evidence. The near-constant settings op1 and op2 fluctuate around a single value without forming separate operating regimes, which is consistent with the single sea-level condition. The decision on s6 does not establish that it is unrelated to remaining life; it records a preprocessing assumption based on the restricted variation observed.

Table 5.1. Variability screening of the twenty-four candidate variables.

Group	Variables	Evidence	Decision
Constant	op3, s1, s5, s10, s16, s18, s19	One recorded value across 20,631 observations	Remove
Near-constant settings	op1, op2	Standard deviations of 0.0022 and 0.0003, with no distinct operating regimes	Remove
Near-constant sensor	s6	Standard deviation of 0.0014, two recorded values, range of 0.010	Remove
Candidate sensors	s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, s21	Recorded variation retained after initial screening	Analyse further

The physical definitions given earlier give context for some of these channels. Sensors s1 and s5 measure conditions at the fan inlet and depend on the external operating condition, while s18 and s19 contain demanded fan-speed quantities rather than hardware response. For s10 and s16 the data establish only that neither varies at the recorded precision. These interpretations apply to FD001 alone.

Guilherme reported the same ten removals for FD001 using a variance threshold of 0.001 [20]. The agreement is an external consistency check rather than independent evidence, since neither procedure consults the prediction target.

The fourteen surviving sensors were then examined for redundancy through the pairwise Pearson correlation matrix of the training data. The identifier columns are excluded, and the blank regions are the constant variables removed above, which have no defined correlation with any other. The analysis is descriptive: the coefficients are computed over all 20,631 rows, and rows belonging to one engine are not independent, so they do not support significance statements. The matrix is shown in Figure 5.2.

One pair stands out. Sensors s9 and s14 correlate at $r = 0.963$, the only pair whose absolute correlation exceeds 0.95; the next highest is s11 with s12 at $r = -0.847$. The threshold of 0.95 is adopted from the comparable FD001 study of Guilherme [20], and the gap between 0.963 and 0.847 makes the decision insensitive to the precise cut-off.

The overlap has a physical explanation. As documented in Section 4.4, s9 records the physical core speed and s14 the corrected core speed, which expresses the speed a component would attain if its inlet temperature corresponded to standard sea-level conditions, obtained by dividing the measured speed by the square root of the ratio of inlet temperature to that reference [7]:

$$NR_c \propto \frac{N_c}{\sqrt{T_2/T_{\text{ref}}}}, \quad \text{where } T_{\text{ref}} = 518.67 \text{ }^\circ\text{R} = 288.15 \text{ K.}$$

The inlet temperature enters through s1, established above as constant in FD001. When that temperature does not vary, the divisor is constant too and the corrected speed becomes the physical speed multiplied by a fixed factor, so the two sensors are related by a linear rescaling. The observed correlation is 0.963 rather than exactly 1.00; this residual may reflect the

measurement noise the simulator adds to its output channels and other transformations applied within it [3], and it is not interpreted further.

Of the two, s9 is retained: their information overlaps almost entirely under a single operating condition, and s9 is directly measured whereas s14 is derived from it and from a second sensor. Guilherme makes the same choice [20]. Asif et al. retain both in a fourteen-sensor set [6], a difference following from their criterion, which evaluates each sensor against the remaining-life target rather than against the other sensors.

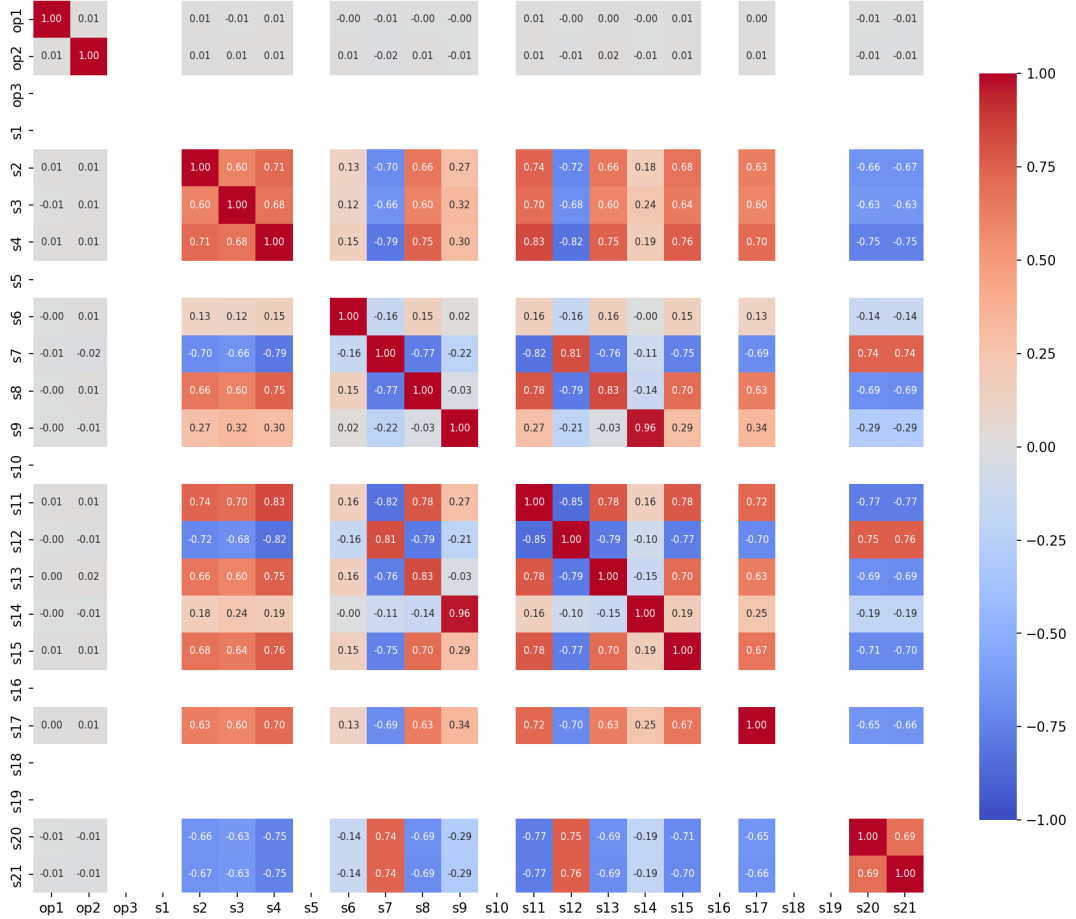


Figure 5.2. Pearson correlation matrix of the FD001 training data, excluding the identifier columns. Blank rows and columns correspond to the variables that show no variation within FD001: a variable with zero variance has no defined correlation with any other, so no coefficient exists for it. The only pair exceeding an absolute correlation of 0.95 is s9–s14, at $r = 0.963$.

Thirteen sensors therefore remain, and they constitute the feature set examined in the remainder of this chapter. They are listed in Table 5.1 and again in Section 6.1, where they enter the models.

5.4 Sensor Behaviour over an Engine's Life

The thirteen retained sensors are examined here for whether they carry a degradation signal, how that signal is shaped over a trajectory, and how far it varies between engines. Each question bears on a different aspect of how a history should be summarised. Figure 4.2 already showed four of them for three engines; the analyses below extend that view to all thirteen and to all 100.

5.4.1 Trend Direction and Its Consistency

A trend visible in an average over 100 engines is compatible with all of them moving the same way, and equally with a minority moving the other. Two analyses were therefore carried out: the first aggregates across engines to establish direction, the second examines each engine separately to establish consistency.

For the fleet-level analysis, every cycle was labelled with its uncapped linear remaining life and sensor values were averaged across engines at each distance from failure, over the final 200 cycles. Aligning by remaining life rather than elapsed cycle is necessary because engines fail at different ages (Section 5.2).

The alignment uses the linear target of Equation (2.1) rather than the piecewise form, which would collapse all early cycles onto a single value and remove the distinction being examined. The procedure is possible only for training engines, whose failure cycle is known (Section 4.5). Because engines differ in length, the number contributing is not constant across the range: all 100 contribute up to a remaining life of 127, whereas at 200 only the 46 engines surviving that long are represented, so the curve at larger values describes a progressively smaller subset.

Nine of the thirteen sensors rise as failure approaches, namely s2, s3, s4, s8, s9, s11, s13, s15, and s17, while four fall: s7, s12, s20, and s21. The fleet means for all twenty-one channels, with the spread across engines, are given in Figure A.1. The seven variables that remain flat are those already removed in Section 5.3, which is consistent with that decision without independently confirming it.

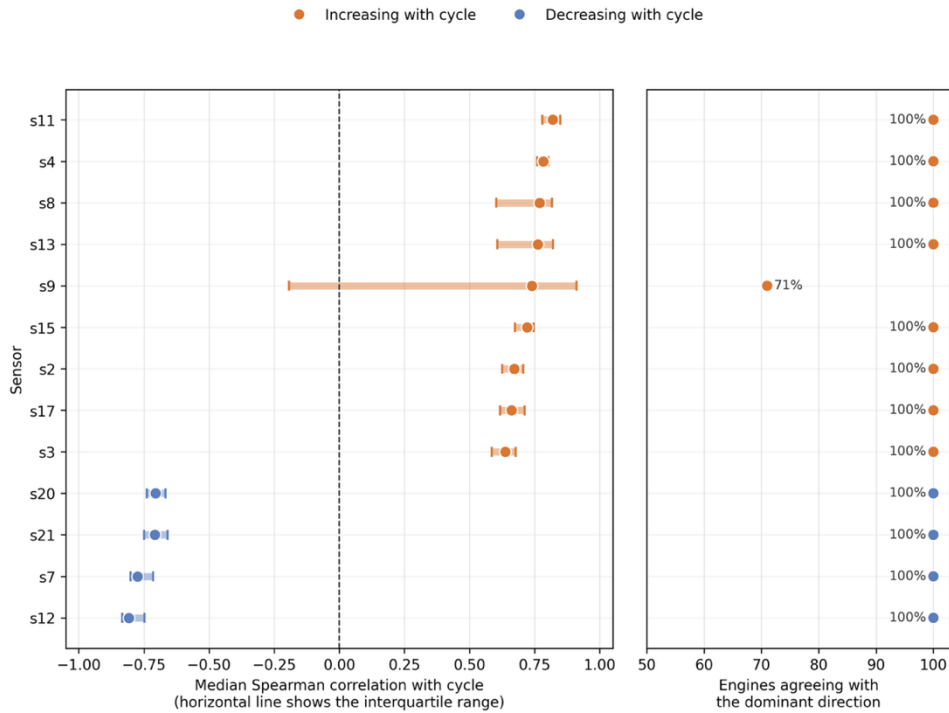


Figure 5.3. Consistency of sensor trends across the 100 training engines, measured by the Spearman rank correlation between each sensor and the cycle index within every engine. The left panel gives the median correlation per sensor, with the horizontal line spanning the interquartile range across engines; orange marks sensors that increase with cycle and blue those that decrease. The right panel gives the percentage of engines whose correlation carries the dominant sign.

For each engine individually, the Spearman rank correlation between each sensor and the cycle index was computed, giving 100 values per sensor. The coefficient measures whether a sensor tends to move in one direction over an engine's life as a whole, not whether it does so at every step.

Figure 5.3 summarises these correlations. For twelve of the thirteen sensors, all 100 engines agree on direction. The median correlations range in magnitude from 0.637 for s3 to 0.819 for s11, and their interquartile ranges are narrow, so the strength as well as the direction varies little across the fleet.

Sensor s9 is the exception. Only 71 of the 100 engines follow its dominant direction, and its interquartile range extends from -0.193 to 0.911 , so in some engines the association is absent or reversed while in others it is strong, and its median of 0.739 is not representative. It is retained nonetheless: it was kept on a redundancy criterion, and an inconsistent direction is not treated here as grounds for removal. The behaviour is also physically plausible, since s9 records the physical core speed while the module that degrades in FD001 is the high-pressure compressor, on the same spool and under a closed-loop controller. Section 7.5 returns to both points.

5.4.2 Non-Linear Shape

A direction says nothing about pace. A sensor moving steadily throughout an engine's life and one barely moving before changing rapidly near failure would both register as increasing, and which of the two describes FD001 determines whether a single slope can represent a trajectory.

For each engine, the cycle index was rescaled to the interval from zero to one so that trajectories of different lengths become comparable, and three quantities were computed per sensor: the slope over the first third, the slope over the last third, and the coefficient of the quadratic term fitted over the whole record, which measures how far the trajectory departs from a straight line. At a constant pace the two slopes would be similar and the quadratic coefficient close to zero; if the pace increases, the late slope will be the steeper and the quadratic coefficient will carry the sign of the overall trend. Steeper here means a larger magnitude in the sensor's own direction: a larger positive slope for a rising sensor, a more negative one for a falling sensor.

Figure 5.4 shows the same phenomenon across all thirteen sensors at once. The pattern is uniform: the deviation from each engine's starting level remains small over the first half of life and grows sharply over the final thirty per cent. The direction differs between sensors, but the timing does not.

Averaged across the fleet, s11, one of the sensors associated directly with the degrading component (Section 4.3), has a mean slope of 0.142 units per normalised lifetime over the first third against 1.786 over the last, a ratio of 12.6 . The corresponding ratios for s4 and s17 are 12.5 and 15.0 . These describe the typical case rather than any individual engine.

The consistency of the effect is as notable as its size. For twelve of the thirteen sensors, every one of the 100 engines shows the steeper slope over the final third, and for the same twelve the quadratic coefficient carries the sign of the overall trend in all 100. The acceleration is therefore a property of each individual trajectory, not only of the fleet average. Sensor s9 is again the exception, with 70 of 100 engines showing the steeper late slope and 68 of 100 a quadratic coefficient of consistent sign.

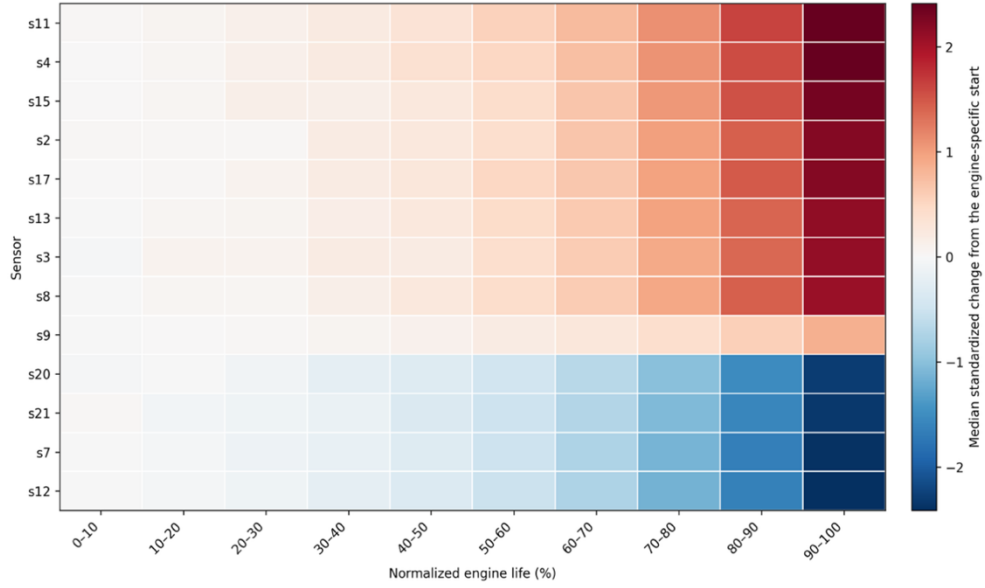


Figure 5.4. Median standardised deviation of each sensor from its own engine-specific starting level, across the 100 training engines. The cycle index is rescaled so that zero per cent is the first recorded cycle and one hundred per cent failure, and each sensor is expressed in units of its own standard deviation. Red indicates an increase relative to the starting level and blue a decrease.

Rescaling time to the unit interval requires knowing each trajectory's full length, which is available only because these are training engines recorded to failure (Section 4.5). The analysis is therefore retrospective, and any feature derived from it must be computed instead from the history available at the prediction cycle, as Section 6.4.2 sets out. The quantities also characterise the recorded signals, not the underlying fault: a sensor may respond non-linearly to a fault growing at a steady rate, so acceleration in the measurements does not establish that the mechanism accelerates. A single slope fitted over a whole trajectory therefore describes neither phase, which motivates testing an explicit description of curvature among the candidate features (Section 6.4.4).

5.4.3 Between-Engine Variation

Whether the differences in initial condition and lifetime established in Sections 1.2 and 5.2 are visible in the sensor measurements, and whether they persist towards failure, is examined next.

For each engine and each retained sensor, an initial level was computed as the mean of the first five recorded cycles and a final level as the mean of the last five. Averaging five observations limits the influence of individual noisy readings while keeping the measurement close to each end of the trajectory. The ratio of the between-engine standard deviation at the end to that at the start summarises how the spread of the fleet changes.

Three descriptive bands are used, fixed before the results were examined: below 0.80 the fleet becomes more alike, between 0.80 and 1.25 the spread is largely unchanged, and above 1.25 the fleet becomes more dispersed. These are conventions adopted for description and carry no statistical status.

Sensor s11 illustrates the typical case. The spread between engines over their first five cycles is of the same order as the mean change of 0.796 psia that a typical engine undergoes over its whole life, and by failure that spread has fallen to 0.352 of its initial value.

Convergence is not universal. Ten of the thirteen sensors fall in the converging band, with ratios from 0.352 for s11 to 0.760 for s3. Two, s8 at 1.099 and s13 at 1.151, fall in the largely unchanged band. Sensor s9 stands far outside both at 7.083: its between-engine standard deviation is seven times larger at failure than at the start, so engines beginning at similar core speeds end at markedly different ones. Figure 5.5 places every sensor in one of the three bands.

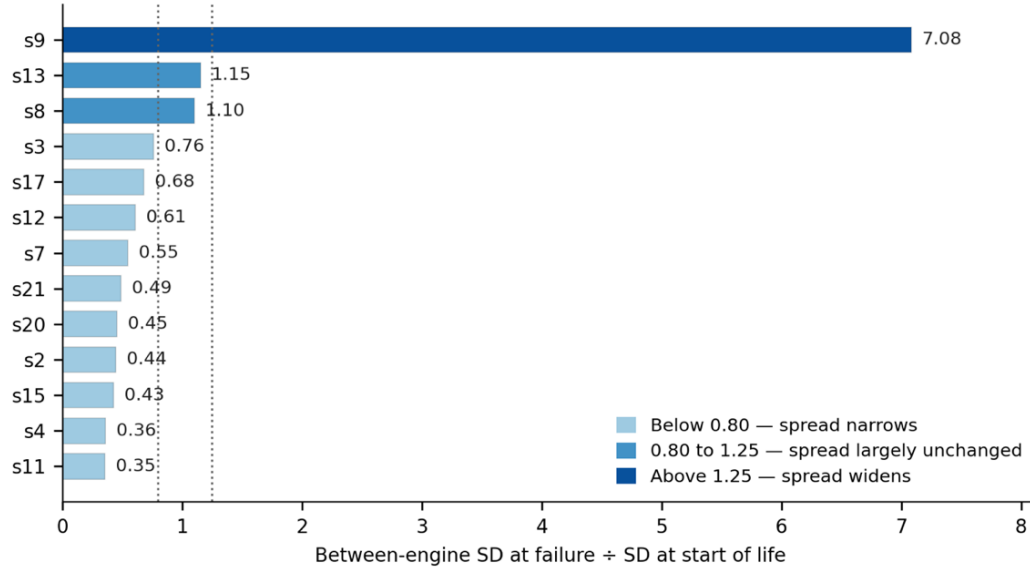


Figure 5.5. Ratio of the between-engine standard deviation at failure to that at the start of life, for each retained sensor. Shading indicates the band into which each sensor falls, from the lightest for a narrowing spread to the darkest for a widening one; the dotted lines mark the band limits of 0.80 and 1.25.

This sits alongside the inconsistent trend direction reported in Section 5.4.1. That s9 is unusual under both measures indicates that it responds to degradation in a way that differs substantially from engine to engine. Section 7.5 takes this up alongside the permutation result for the same sensor.

Because two engines may record similar current values having started from different levels, the latest reading alone does not describe how an engine reached its state; displacement from its own starting level, and the rate at which that displacement accumulated, may add to what it conveys. Whether they do is tested in Section 6.4.1.

5.5 Properties of the Recorded Signals

The preceding section described how sensors behave. This one addresses two properties of the records themselves that bear on how they may be used: whether each trajectory is continuous, and where extreme values occur within it.

A trajectory may be an uninterrupted progression towards failure, or it may contain points at which an engine's condition was restored, for example by a repair. Remaining life decreases by construction either way (Section 4.5). What an intervention would break is the correspondence between the readings and that quantity: the same values would then sit at two different distances from failure within one record.

The dataset documentation states that the fault grows until failure and describes no intervention [3]. This was verified against the data rather than adopted from the documentation alone. Sensor

s11 was used, for the reason given in Section 5.4.2, and three quantities were computed for each of the 100 engines on a smoothed version of the signal: the net rise over the trajectory, the largest fall from any running peak, and the proportion of cycles on which the smoothed value increases. The procedure, the two smoothing settings examined, and the full figures are given in Appendix B.

Neither flag condition was met by any engine. The net rise was positive for all 100, and the largest drawdown in any single engine was an order of magnitude smaller than its own rise. This does not show that s11 rises monotonically, since roughly a third of the smoothed steps fall, but that no engine shows a large sustained reset at a twenty-cycle resolution. The trajectories can therefore be treated as uninterrupted runs to failure, and no feature for time since an intervention is required.

A routine step in preparing tabular data is to remove observations lying far from the bulk of a distribution. The question here is not whether such values exist but where: distributed evenly across engine life they are plausibly measurement artefacts, but concentrated at a particular stage they coincide with the phenomenon the models are meant to detect. For each of the thirteen sensors, values were flagged by the conventional interquartile criterion, which treats an observation as extreme if it lies more than 1.5 interquartile ranges below the first quartile or above the third.

The quartiles were computed from the training data alone, on the pooled distribution of each sensor across all cycles and engines. Each flagged observation was assigned to a ten-cycle interval by its uncapped linear remaining life, and results are reported as proportions rather than counts. The distinction does not alter the outcome here: because every engine survives at least 128 cycles (Section 5.2), all 100 contribute to every interval up to a remaining life of 119, so each of the twelve intervals contains exactly 1,000 observations.

Figure 5.6 illustrates the resulting pattern for one sensor; the range across the thirteen is given below.

The flagged observations are not evenly distributed. For every sensor, the proportion is highest in the interval immediately preceding failure and falls sharply thereafter. In the final ten cycles it ranges from 6.2 per cent for s17 to 43.7 per cent for s9, with most sensors between 8 and 15 per cent; by a remaining life of fifty cycles it has fallen to zero or near zero for every sensor except s9, which retains 7.2 per cent.

This follows from the shape established in Section 5.4.2: the readings lying furthest from the pooled distribution are those recorded when the change is most advanced. That pooled distribution is dominated by earlier-life cycles, simply because most cycles belong to that part of a trajectory. Readings from the final stretch are therefore flagged not because they are anomalous but because they differ systematically from the bulk of the record.

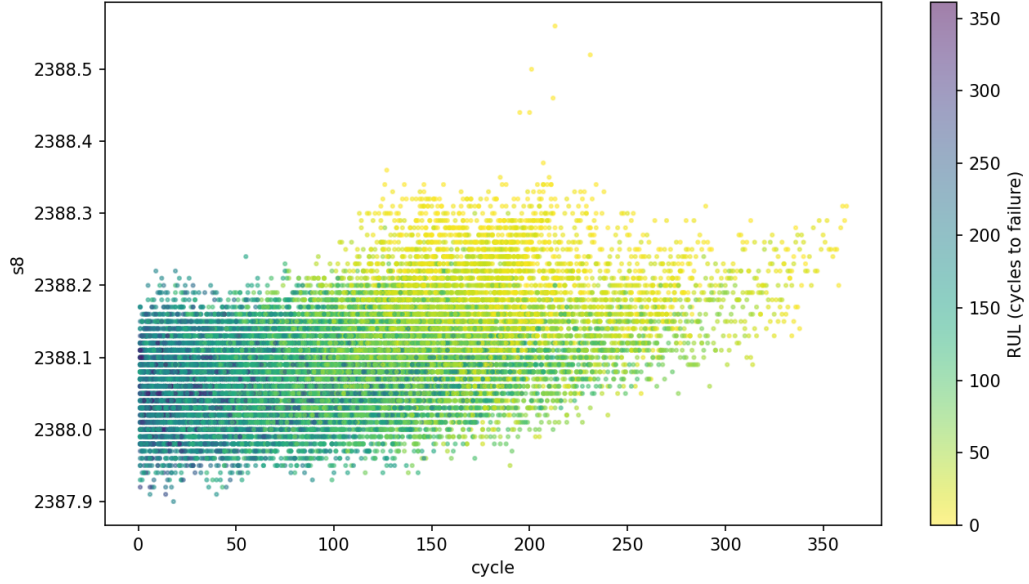


Figure 5.6. All recorded values of sensor s8 across the 100 training engines, plotted against cycle index and coloured by remaining useful life, from dark purple far from failure to yellow close to it.

The criterion is inappropriate for signals of this kind, because it treats all observations of a sensor as drawn from one fixed distribution, whereas here the distribution shifts systematically over an engine's life. A rule removing the flagged observations would discard late-life cycles far more often than early-life ones, acting on the phase in which prediction matters most. No outlier removal is applied at any stage of this work.

The analysis establishes where flagged values occur, not whether any individual one is a genuine measurement error; some proportion may be, and nothing here demonstrates that a flagged value is a valid reading. And s9 is again distinctive, showing elevated proportions at distances from failure where the others show none.

5.6 Detectability of an Early-Life Signal

This section tests the proposition of Sections 1.2 and 2.3, that degradation becomes visible in the sensor signals only after an engine has operated for some time, by asking how much of an engine's history must be observed before its total lifetime can be predicted from the sensors alone. Under the linear target of Equation (2.1), remaining life at a given cycle is the total lifetime minus that cycle index, which is known, so the two problems are equivalent; the equivalence does not extend to the piecewise target, whose value is bounded.

For each window length N , features were formed from the thirteen retained sensors over the first N cycles of each engine, taking the mean, standard deviation, and linear slope of each to give thirty-nine features. A ridge regression with a fixed regularisation strength of one, preceded by standardisation, was fitted within a five-fold cross-validation over the 100 training engines, with both steps refitted on the training portion of every fold, so that no engine contributes to the model that predicts it. The sensor set itself was fixed beforehand from the full training data (Section 5.3) and is therefore common to all folds. Window lengths were fixed in advance as a regular grid from 10 to 120 cycles in steps of ten.

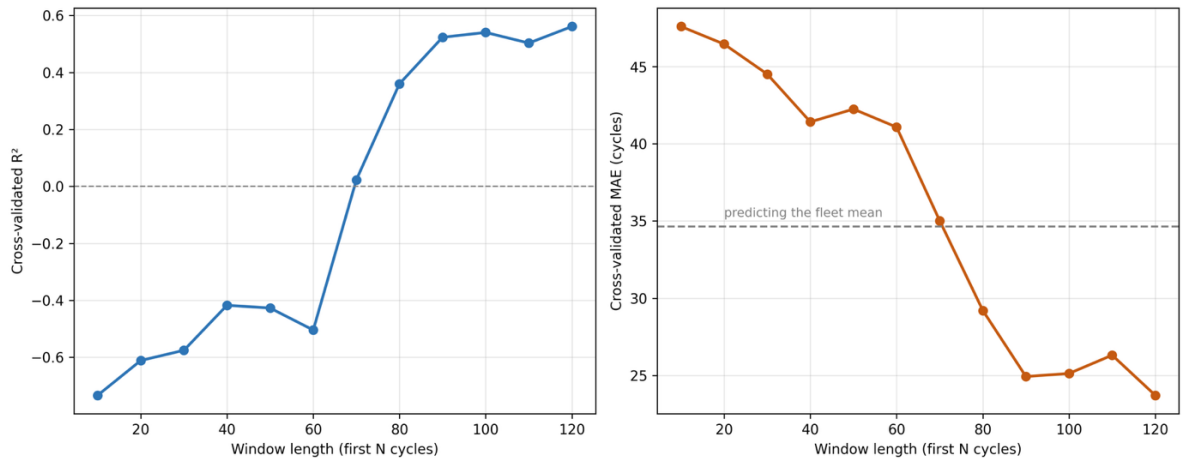


Figure 5.7. Cross-validated performance in predicting total engine lifetime from the first N cycles, for window lengths from 10 to 120 cycles. The left panel gives the coefficient of determination, with the dashed line at zero marking the level of a constant prediction; the right panel gives the mean absolute error, with the dashed line marking the error obtained by predicting the fleet mean.

The upper limit follows from the shortest trajectory of 128 cycles (Section 5.2). Every window is then available for every engine, so the sample does not change across window lengths. Two references are used. Predicting the fleet mean for every engine gives a mean absolute error of 34.6 cycles, and the coefficient of determination is defined against this constant. Predicting the fleet median gives 33.7 cycles and is the stricter reference, since it minimises absolute error.

Up to and including sixty cycles the coefficient of determination is negative at every window and the error exceeds both references, so the model predicts worse than the constant on held-out engines. The rank correlation stays weak, and with twelve windows none of these correlations meets the Bonferroni-corrected threshold of $0.05 / 12$, or 0.0042. The transition falls between sixty and seventy cycles, and the two criteria do not move together. At seventy the rank correlation clears the threshold, yet the error of 35.0 cycles still exceeds both references. The first improvement comes at eighty cycles, where the error falls to 29.2; the association reaches 0.835 at 120. Figure 5.7 gives the full curves. The error does not fall evenly over the upper range, but these fluctuations were not tested against run-to-run variability and are not interpreted.

A second test took a different route. Engines were split into quartiles by lifetime, and each of the thirty-nine features was compared between the shortest- and longest-lived quartiles at a window of fifty cycles, using the Mann-Whitney U test of Section 2.6. The corrected threshold is $0.05 / 39$, or 0.00128, and no feature separated the groups. This agrees with the negative coefficient of determination at that window.

The finding is that a usable predictive signal becomes detectable after about eighty observed cycles. It is not that degradation begins there: smaller differences may be present earlier, but under this representation and this model they cannot tell one engine's eventual lifetime from another's. The first claim is about what a predictor can use, the second about the physical process; only the first is supported. The longer windows are not confined to early life: 120 cycles covers 94 per cent of the shortest trajectory, so part of the gain at the upper end comes from the window nearing the end of those engines' lives. The windows are also defined by cycle

index, while the target constant of Section 5.7 is defined by remaining life, so the two should not be equated.

5.7 Target Definition and Modelling Implications

This section turns from the sensor data to the label, quantifying what the two definitions of Section 4.5 imply for the FD001 training set. All figures are computed over rows rather than engines, so a long-lived engine carries more weight than a short-lived one (Section 5.2). The section closes by collecting the findings of the whole chapter and the decisions they constrain.

Under the linear definition of Equation (2.1), the target at an engine's first cycle equals its lifetime minus one. Applying the piecewise definition of Equation (2.2) with $R_{\max} = 125$ lowers the maximum and the mean but leaves the median unchanged, which indicates that the constraint acts on one side of the distribution only.

Table 5.2. Distribution of the training target under both definitions.

Statistic	Linear	Piecewise ($R_{\max} = 125$)
Maximum	361	125
Mean	107.8	86.8
Median	103	103
Rows altered	—	8,031
Proportion of rows altered	—	38.9%
Rows equal to 125 after capping	100	8,131
Engines affected	—	100

The two counts in Table 5.2 differ because 100 rows already carried a target of exactly 125 before capping, so the number of rows equal to 125 afterwards exceeds the number altered. Every engine contributes at least one altered row, which follows from the lifetime distribution: even the shortest reaches a linear target of 127, so no trajectory escapes the constraint.

The altered rows are, by definition, those furthest from failure, which is the region of lowest average rate of change (Section 5.4.2). Capping therefore changes what a model is asked to learn over more than a third of the data, which is why the target definition is treated here as an experimental variable rather than a fixed preprocessing choice. The value of 125 originates in later studies on the C-MAPSS subsets, not in Heimes's paper, which adopted 130 (Section 2.3); Section 6.3 gives the reasoning behind it.

Each analysis above establishes a property of FD001 that constrains a decision taken in the chapters that follow. Table 5.3 collects them.

Table 5.3. Findings of the exploratory analysis and the decisions they inform.

Finding	Evidence	Implication for the modelling design
No missing, duplicated, or malformed observations	Four integrity checks, Section 5.1	No imputation or row removal during cleaning

Finding	Evidence	Implication for the modelling design
Engines fail at widely differing ages	Lifetimes from 128 to 362 cycles, Section 5.2	Cycle index alone cannot locate an engine within its degradation; long engines carry more weight in row-level statistics
Ten variables show no usable variation in FD001	Constancy, cardinality, and range, Section 5.3	Removed before modelling
Two sensors carry strongly overlapping information	s9 and s14 at $r = 0.963$, Section 5.3	s14 removed; thirteen sensors retained
The retained sensors trend consistently within engines	100% direction agreement for twelve of thirteen, Section 5.4.1	The retained set is carried forward; predictive value is assessed in Chapter 7
Sensor s9 behaves differently from the rest	71% direction agreement; spread ratio 7.083, Sections 5.4.1 and 5.4.3	Noted as an anomaly rather than removed; discussed in Chapter 7
The rate of sensor change increases over a trajectory	Late-third slope steeper in 100 of 100 engines for twelve sensors, Section 5.4.2	A single slope cannot describe the shape; curvature included among the candidate features and tested
Engines start at different levels and change at different rates	s11 initial span 0.60 psia against a mean change of 0.796, Section 5.4.3	The latest reading alone does not describe how an engine reached its state; history representations compared
No large sustained reset in sensor s11	No engine flagged on either criterion at a twenty-cycle resolution, Section 5.5	Records treated as uninterrupted runs to failure; no intervention feature required
Extreme values concentrate close to failure	Flagged proportion highest in the final interval for every sensor, Section 5.5	No outlier removal
Lifetime not recoverable from the earliest cycles by the method tested	Negative R^2 up to 60 cycles, Section 5.6	Motivates testing a target that does not vary in that region
Capping alters more than a third of the training rows	8,031 of 20,631 rows, Section 5.7	Target definition treated as an experimental variable

Several of the findings are narrower than the table's phrasing suggests. The continuity result rests on one sensor at one smoothing resolution, and the early-life result on one set of features and one model. Consistent trending shows that a sensor moves systematically with degradation, but not that it contributes to a prediction; that is settled by the experiments of Chapter 7, not by description. Chapter 6 sets out the experimental design that proceeds from these properties.

6 Methodology

This chapter describes the design of the experiments rather than their outcome. Intermediate validation results appear where they justify a decision: why one representation was preferred over another, or why a preprocessing step was rejected. The performance of the four models on the test set is reported in Chapter 7.

6.1 The Prediction Task and Data Preparation

The quantity predicted is the remaining useful life in flight cycles, and its estimation is a regression problem. The two candidate definitions of this target were set out in Section 2.3; the choice between them is made in Section 6.3.

Training and evaluation differ in how many predictions each engine contributes. Every cycle of a training trajectory can serve as a training example (Section 4.2): the model sees the history up to that cycle and is asked for the remaining life at it. This yields 20,631 available engine-cycle pairs. Evaluation, by contrast, consists of exactly one prediction per test engine, made from that engine's full observed history and compared against the single ground-truth value supplied for it.

The test input file contains 13,096 rows, but these are the recorded cycles of the test engines, not 13,096 separate prediction tasks: the earlier cycles of a test engine are input, not evaluation points. The figures reported in Chapter 7 are computed over 100 predictions, one per engine.

The training samples have two properties that come from the data itself rather than from any design choice: they are not statistically independent, since consecutive cycles of one engine describe the same trajectory, and longer-lived engines contribute proportionally more of them (Section 5.2). Both bear on how the data may be partitioned, which Section 6.2 addresses.

What constitutes a single input differs between the two model families. The tabular models require a fixed-length vector and therefore receive a summary of an engine's history, whereas the sequence models receive the ordered measurements themselves. Section 6.4 specifies both representations. The operations described here are applied to all four models. Any quantity learned from the data is estimated separately from the relevant training partition, for the reason given in Section 6.2.2.

Eleven of the twenty-four candidate columns are removed on the evidence of Section 5.3: ten that show no usable variation within FD001, and s14 as redundant with s9. Thirteen sensors remain: s2, s3, s4, s7, s8, s9, s11, s12, s13, s15, s17, s20, and s21.

Two further columns are retained but are not predictors. The unit identifier serves as a grouping key, used to construct each engine's history and to keep engines separate when the data are partitioned; the cycle index orders the observations within an engine. Neither is supplied to a model as an input variable.

The retained sensors are transformed using min-max scaling, which maps the training range of each sensor onto the interval from zero to one. Within each cross-validation fold the scaler is fitted only on the engines assigned to the training partition and then applied unchanged to the held-out engines; once the design was fixed, a final scaler was fitted on all 100 training engines and applied unchanged to the test set. Held-out or test values may therefore fall outside the unit

interval where they exceed the corresponding training extrema. The transformation is required for the two neural models, whose optimisation is sensitive to input magnitude, and it gives all four models a common input scale. The tree ensemble is unaffected by it (Section 2.4.2).

No observations are removed, for the reason established in Section 5.5. All 20,631 training rows and all 13,096 test rows are retained.

The cycle index is not supplied as a feature, since it does not on its own indicate how far degradation has progressed (Section 5.2). Excluding it directs the models towards the condition measurements themselves.

The exclusion does not affect the two model families identically. The tabular models receive no explicit variable describing the number of observed cycles: histories of thirty and of three hundred cycles both produce a vector of the same length, although some summary statistics may still carry indirect information about the span they were computed over. The sequence models receive one ordered time step per observed cycle and can therefore recover the length from the valid portion of the sequence or from its padding pattern. This follows from the representations themselves rather than from a design choice.

6.2 Experimental Protocol

Every partition used to produce the reported validation results and final models is made over engines rather than over rows, whether it is a cross-validation fold or the inner split used for early stopping. An engine is assigned entirely to one side of a partition, never divided across it.

Consecutive cycles of one engine describe the same trajectory at successive moments, so a model that has seen the first half of an engine's record has seen the conditions under which its second half unfolds. A partition made over rows would place such cycles on both sides, and the resulting estimate would measure how well a model interpolates within a known trajectory rather than how well it generalises to an unseen engine. The training and test files contain distinct simulated units, although unit identifiers are numbered independently within each file.

6.2.1 Validation Design

Validation uses five-fold cross-validation with the engine identifier as the grouping key, applying the principle stated above: all cycles of an engine fall in the same fold.

Within each fold, the twenty validation engines are scored at four fixed points: the cycles at which their remaining life first falls to 110, 80, 50, and 20. These values were specified before the experiments were run and are not derived from the test data. The four locations are defined using the uncapped linear remaining life, irrespective of which target definition is being evaluated, so that the observed histories are identical when different caps are compared.

Each fold therefore contributes eighty evaluation points, and the five folds give 400 in total. The mean absolute error is computed over the eighty points of a fold; the reported validation figure is the mean of the five fold-level values, and the accompanying standard deviation describes the variation between folds.

The validation engines are scored at truncation points rather than at their final cycles, because a training engine runs to failure and its last cycle has a remaining life of zero; evaluating there would test whether a model can recognise an engine that has already failed. The same folds and

evaluation points are used for every experiment in this chapter, so any difference between two configurations reflects the configurations themselves rather than a difference in how they were assessed.

Where a neural model requires early stopping, the engines used for it are drawn explicitly from the current fitting partition. A fixed set is reserved before training and contributes no sample to fitting: twenty engines for the LSTM and ten for the Chronos-2 regression head. The same fitting and inner-validation engines are retained across seeds, in the sense of Section 2.2, so variation arising from the data partition is held fixed while the stochastic components of training, namely weight initialisation, dropout masks, and batch ordering, are allowed to vary.

The inner-validation engines are excluded from fitting and the model is not subsequently refitted on the complete training set. In the final fit from all 100 training engines this leaves eighty for the LSTM and ninety for the Chronos-2 head, against one hundred for the tabular pipelines; within a cross-validation fold, whose fitting partition already contains eighty engines, the corresponding counts are sixty and seventy. This is the cost of partitioning by engine at this stage. The alternative, selecting a stopping epoch on the inner split and then refitting on all engines for that fixed number of epochs, was not carried out.

6.2.2 Separation of the Training and Test Data

Any quantity estimated from data can carry information about the partition it was estimated on. Two such quantities occur here: the min-max scaler applied to the sensor values and the standardisation applied to the Chronos-2 embeddings. Both are fitted on the fitting partition alone and applied unchanged elsewhere, following the procedure of Section 6.1, and code-level checks confirmed this for each transformation.

No input to any model draws on a cycle later than the one being predicted. For the tabular pipelines this is enforced by the construction described in Section 6.4.2; for the sequence pipelines, the sample at cycle t is the sequence of the first t cycles, and the remainder of the record is not read. The same construction is applied to training and test data. One quantity is an acknowledged exception to fold-specific estimation: the padded sequence length of 362 cycles was fixed once from the official training set before cross-validation rather than recomputed within each fold. It involves no test data, being the length of the longest training trajectory (Section 6.4.3).

Two departures from this protocol occurred during the work and are recorded here, since both affect how the results of Chapter 7 should be read.

The first departure concerns the scope of the validation protocol. In an initial round of experiments, model configurations were selected on an engine-level holdout, twenty training engines set aside before fitting and used as the validation set of Section 2.2, but three choices that shape the method more fundamentally than a hyperparameter, namely the value of the target cap, the representation of the history, and whether to smooth the signals, were compared by their effect on the test set instead. A decision taken because it improved the test error is a decision the test set has informed, and any subsequent measurement on that same set inherits the optimism this introduces. The model configurations selected by the original holdout were retained, since that selection had used held-out engines rather than the test set.

The second departure concerns the early-stopping split. This was originally specified as a fraction of the training array rather than as a set of engines. The framework takes such a fraction as the final rows of the array in their given order; because the training samples are ordered by engine, the resulting boundary falls at a row index that need not coincide with the end of an engine's record, so one engine could contribute its earlier cycles to fitting and its later cycles to the portion used for stopping. This is the situation the partitioning rule stated at the start of this section is meant to prevent. The split was redefined over engines as specified in Section 6.2.1, and both neural models were retrained; the results reported in Chapter 7 are those of the corrected version.

All three comparisons were re-run under the training-only protocol of Section 6.2.1, though re-running them does not remove the optimism recorded above. The pipeline was then frozen, after which the official test engines were used for two purposes only: the final evaluation reported in Chapter 7, and the post-hoc permutation inspection of Section 7.5. Neither informed a design decision, since the inspection was carried out on the frozen pipeline and altered nothing in it. Table B.2 summarises the checks.

6.3 Definition of the Target

Section 2.3 introduced the two ways of defining the target and Section 4.5 gave their form for this dataset. What remains is to choose between them and to fix the constant the piecewise form requires.

The piecewise form is adopted, on the grounds set out in Section 2.3 together with the evidence of Section 5.6, where no usable early-life signal could be recovered under the representation and predictor tested there. This supports treating the early portion of a trajectory as a region of constant label, as a modelling approximation, though it does not locate the physical onset of degradation or determine the value of the constant.

The constant presents a harder problem than it appears. Caps of 100, 125, and 130 were compared using XGBoost with the raw sensor values as input, representation A of Section 6.4.1, under the protocol of Section 6.2.1. Each model was trained on its own capped target and evaluated against the unrestricted remaining life, so that all three were judged on the same quantity. The mean absolute error was 14.38 ± 1.59 cycles at a cap of 100, 15.66 ± 2.38 at 125, and 16.42 ± 2.54 at 130.

These figures document what each cap costs in predictive terms; they do not determine which cap is appropriate. A lower cap deliberately removes distinctions among observations far from failure and is therefore disadvantaged when its predictions are judged against uncapped values above that cap. Evaluating each model against its own capped labels instead changes the quantity and the range over which the error is measured, so the errors are not comparable across caps, and the ordering among the caps changed when the experiment was run that way. The analysis was therefore used to document the consequences of the choice rather than to make it.

The value of 125 is adopted on structural grounds. Section 5.2 established that the shortest training engine reaches a maximum linear target of 127. Any value below 127 alters at least one label in every training trajectory; at 125 the shortest engine has a three-cycle plateau, although only its first two labels differ numerically from the linear target, since the third already equals 125. A value of 127 or above would leave that trajectory unchanged. The value is also

commonly used in the C-MAPSS literature, which keeps the results reported here comparable with published work. The distinction from Heimes's own value of 130 is recorded in Section 5.7.

For the two neural models the target is divided by a constant before training and the predictions are multiplied by the same constant afterwards: 125 for the piecewise target, and 361 for the linear target, this being the largest linear value in the training set. Scaling keeps the quantity being optimised on a numerical range comparable to that of the scaled inputs, and it is arithmetic: it does not alter what the model is asked to predict. Without it, the first attempt to train the LSTM produced predictions that varied little between engines, while the validation loss fell only slightly before flattening; optimisation became stable once the target was scaled. A squared error measured on a range of 125 produces gradients much larger than one measured on the unit interval, which is the range the weight initialisation assumes, but the experiment does not isolate this as the sole factor. The comparison between the two target definitions is reported in Section 7.4.

6.4 Representation of the Observed History

6.4.1 Equalising the Observation Horizon

The four models differ in architecture, which is the intended comparison. In an initial round of experiments they also differed in how much of an engine's trajectory each could see: the tabular models received the sensor values of the prediction cycle alone, the LSTM up to fifty cycles, and Chronos-2 up to one hundred. Each setting was reasonable for the architecture in question, and each had been chosen without reference to the others. A difference in error between a tabular pipeline and a sequence pipeline could therefore be attributed to either factor, and the experiment provided no way to separate them.

The tabular models require a fixed-length input vector and do not process variable-length ordered sequences directly, but historical information can be supplied to them through lagged values, fixed-window inputs, or summary features. In the initial experiment no such representation was provided.

The observation horizon, the span of an engine's trajectory a model is permitted to see, must therefore be equalised, which means giving each model family the full observed history, that is the complete record of cycles up to the prediction point, in the form its architecture accepts. The effect of the horizon should also be measured, so that the size of the confounding factor is documented rather than assumed. That effect was measured by an experiment in which the model, the target, and the evaluation protocol are held fixed while the amount of history supplied is varied. Three levels were compared under the protocol of Section 6.2.1:

- A, the sensor values of the prediction cycle alone;
- B, the five summary statistics of Section 6.4.2 computed over up to the fifty most recent observed cycles;
- C, the same five summary statistics computed over the full observed history.

Table 6.1. Validation mean absolute error under three levels of observed history.

Representation	Inputs	Linear regression	XGBoost
A, prediction cycle only	13	16.60	14.29
B, up to fifty recent cycles	65	13.93	8.55
C, full observed history	65	12.97	6.91

The step from A to B is substantial for both models and particularly so for XGBoost, where the error falls by forty per cent. What this step changes, however, is not only the horizon: it extends the history from one cycle to as many as fifty, introduces a statistical summary in place of raw values, and increases the number of inputs from thirteen to sixty-five. The improvement therefore reflects the combined effect of these three changes and cannot be attributed to any one of them.

The step from B to C is narrower in scope, since both use the same five summary statistics and the same number of summary features and differ only in the span over which the statistics are computed. Where an engine has observed fifty cycles or fewer the two are identical, so the comparison concerns only those validation points at which more than fifty cycles had been recorded. The average reduction from B to C was 1.64 cycles for XGBoost and 0.96 for linear regression. For XGBoost the fold estimates at level C varied by 2.65 cycles, which is larger than this reduction. That figure describes the spread of the level-C estimates, not the uncertainty of the paired difference, so the comparison is treated as descriptive.

The full observed history was nonetheless adopted. Its selection does not rest on a demonstrated advantage over the fifty-cycle window, but on the fact that it gives every model family access to the same observation horizon of each trajectory, which is what the comparison requires.

6.4.2 Summary Features for the Tabular Models

The five summary statistics of Section 2.2, namely the mean, the standard deviation, the least-squares slope, the last observed value and the delta, are computed for each of the thirteen retained sensors, giving sixty-five summary features.

Each of the five carries a different aspect of an engine's record. The mean describes the level at which a sensor has operated, and Section 5.4.3 showed that engines begin at measurably different levels, so this quantity distinguishes engines that have operated in different regions of the sensor's range. The standard deviation describes how much a signal has fluctuated: because the retained sensors move systematically over a trajectory (Section 5.4.1), a history spanning a period of change shows greater dispersion than one confined to a stable interval. The slope describes the rate and direction of that movement (Section 5.4.1) and is set to zero where an engine has only one observed cycle. The last value is the quantity the tabular models received in isolation under representation A, retained so that the current state remains directly available alongside the historical summaries. And because engines start from different levels (Section 5.4.3), the delta conveys something the absolute level does not, namely a displacement measured relative to an engine's own starting point.

Every statistic is computed from the observed history up to and including the cycle being predicted, and from no later cycle. For a training engine, the sample at cycle t uses the first t

rows of its record; the remaining rows exist in the file but are not read. For a test engine, the input is the complete record supplied, which by construction ends before failure. The same function performs both computations, so that the difference between the two cases is one of available data rather than of procedure.

6.4.3 Sequence Representation for the LSTM and Chronos-2

The sequence models accept the ordered measurements directly and require no reduction to a fixed-length vector. Engine histories differ in length, so the padding of Section 2.4.3 is applied: shorter sequences are extended with rows of zeros until all reach the length of the longest training trajectory, which is 362 cycles (Section 5.2). This length was computed from the training engines alone; no test sequence exceeds it, the longest being 303. Were a longer sequence to occur, truncation would retain its most recent 362 observations, preserving the cycles closest to the prediction point.

For the LSTM, each training sample is the sequence of sensor readings from an engine's first recorded cycle up to the cycle being predicted, so its length equals that cycle index. The padding is appended after the recorded values, which follows from the implementation: the accelerated recurrent kernel available on the hardware used here operates on right-padded sequences when masking is applied, and a left-padded arrangement falls back to a slower path. A masking layer is placed before the recurrent layer. It marks a time step as padded when all thirteen values at that position are zero, and the recurrent layer then skips that step. Zero-valued time steps are masked, which assumes that no genuine observation has all thirteen scaled sensor values equal to zero (Section 6.1). A mask derived from the recorded sequence lengths would remove that assumption.

Chronos-2 is used as the frozen feature extractor described in Section 2.4.4. It receives the same observed histories and the same maximum length of 362 cycles, but uses zero pre-padding rather than post-padding, and no masking layer is applied.

The model does not return a single vector. For each input it produces a separate representation for every sensor, divided into patches whose number depends on the sequence length, giving an array of sensors by patches by 768 dimensions. These are averaged over both the sensor axis and the patch axis, yielding one 768-dimensional vector per sequence. Averaging within each sensor first and concatenating the thirteen results would give the head $13 \times 768 = 9,984$ inputs and far more trainable parameters; mean pooling keeps 768, at the cost that sensor identity is not preserved as a separate position. The resulting vectors are standardised before the head is fitted, using the procedure of Section 6.2.2.

Without a mask, Chronos-2 has no explicit signal marking which positions are padding. The same padding rule is applied to training and test histories, but this procedural consistency does not remove the possibility that the pooled embedding is influenced by the proportion of padding, which varies with the length of the observed history; a test engine observed for thirty-one cycles is presented as a sequence of 362, of which over ninety per cent is padding.

6.4.4 Additional Representations Examined

Two extensions of the representations above were examined and neither was adopted.

Shape features for the tabular models. The five summary statistics describe level, dispersion, rate, current state, and displacement, but not the shape of a trajectory: a slope fitted over a whole history returns the same value whether the signal moved steadily throughout or barely moved before changing sharply near the end. Section 5.4.2 established that the second pattern is the one FD001 exhibits, so the quadratic coefficient and the slope ratio of Section 2.2 were added to each sensor to test whether an explicit description of curvature improves prediction. The quadratic coefficient is set to zero where fewer than three observations are available. For the slope ratio each third must contain at least two cycles; the ratio is set to zero where the history is too short, or where the early-third slope falls below 10^{-6} in absolute value, and it is clipped to the interval from -50 to 50 . Adding both to each sensor gives ninety-one inputs. The two feature sets were compared under the protocol of Section 6.2.1, on identical rows and folds, the sixty-five-input set being formed by selecting the corresponding columns of the ninety-one-input matrix, so the two contain numerically identical values for the five shared statistics. Table 6.2 reports the validation error of both feature sets.

Table 6.2. Validation mean absolute error with and without the shape features.

Feature set	Inputs	Linear regression	XGBoost
Five statistics per sensor	65	12.97	6.91
With curvature and slope ratio	91	12.82	7.59

The two models moved in opposite directions and by amounts smaller than the variation between folds in either case, so the comparison provides no evidence that the additional quantities improve prediction, and none that they harm it. The sixty-five-input set is retained, on the principle that a representation should not be enlarged without evidence that the enlargement helps.

Length-matched batching for Chronos-2. The padding concern recorded in Section 6.4.3 prompted an exploratory experiment in which sequences were passed at their own observed lengths, grouped into batches of similar length so that padding within a batch spanned a few cycles rather than hundreds, and where padding remained necessary the first observed value was repeated rather than zero inserted.

The alternative pipeline performed worse, giving a mean absolute error of 13.11 cycles against 8.36 for the existing arrangement, both obtained under the same validation protocol with a ridge regression head. Several aspects changed at once: sequence length, batching, padding value, patch count, and the exclusion of one-cycle histories, which left 20,531 rather than 20,631 training windows. The experiment therefore shows that the alternative performed worse as a whole without identifying which change was responsible, and uniform padding was retained. The objection that prompted it stands as a limitation rather than resolved.

All four pipelines therefore receive measurements covering the same observation horizon of each trajectory, while their numerical representations still differ in the ways set out in Sections 6.4.2 and 6.4.3. That remaining difference is the subject of SQ1.

6.5 Final Configuration and Evaluation Protocol

6.5.1 Smoothing of the Sensor Signals

One preprocessing decision remains. Sensor readings vary from one cycle to the next by more than the underlying trend moves in that interval, as Section 5.5 established for one sensor. A moving average would attenuate that variation, and whether doing so helps is an empirical question.

Any such average must be trailing rather than centred. A centred window at cycle t would draw on cycles after t , which is unavailable at the moment of prediction and would violate the causal construction of Section 6.2.2. The average therefore extends backwards only, and the first cycles of a record are averaged over whatever history exists.

Four settings were compared under the protocol of Section 6.2.1, applied to the sensor values before the summary statistics of Section 6.4.2 were computed. Table 6.3 gives the results.

Table 6.3. Validation mean absolute error under four smoothing settings.

Smoothing window	Linear regression	XGBoost
None	12.97	7.02
5 cycles	12.45	7.22
10 cycles	13.04	8.00
20 cycles	13.82	8.20

The unsmoothed figure for XGBoost is 7.02 here against 6.91 in Table 6.1, although both report the same configuration. The two experiments construct the feature matrix by separate code paths, and the difference of 0.11 cycles reflects that rather than a change of setting. It is well within the fold variation and does not affect the comparison, which is internal to this table.

No setting improved on the unsmoothed signals by more than the variation between folds. For XGBoost, however, the error rises monotonically with the window length, and the increase at twenty cycles is seventeen per cent above the unsmoothed value; the individual steps fall within the fold variation, but the pattern across them does not. Smoothing may attenuate information already carried by the dispersion and slope features, particularly during the sharper late-life change identified in Section 5.4.2. The experiment establishes the performance effect rather than its mechanism.

One result runs the other way. At a window of five cycles the PHM score for XGBoost improved from 33.9 to 28.6, computed as the mean of the five fold-level scores, while the mean absolute error worsened. Validation mean absolute error was the prespecified criterion for this decision, so no smoothing was retained; the conflict shows that the decision depends on which aspect of the error is prioritised (Section 2.5). The signals are therefore used as recorded, with no smoothing applied at any stage.

6.5.2 Model Configurations

Table 6.4 gives the configuration of each model. For the three models with settings to select, these were chosen on the engine-level holdout described in Section 6.2.2 and held fixed thereafter. For Chronos-2 the base checkpoint is used rather than the smaller variant the authors also released, since the constraint here is predictive quality rather than inference latency [17]. Row and column subsampling at 0.8 means that each tree is grown on a randomly chosen 80 per cent of the training rows and 80 per cent of the inputs; this decorrelates the trees and is the source of the randomness in XGBoost recorded in Section 6.5.3.

Table 6.4. Model configurations and structural size.

Model	Configuration	Structural size
Linear regression	Ordinary least squares, no regularisation	65 feature coefficients and one intercept, giving 66 fitted parameters
XGBoost	300 trees, maximum depth 8, learning rate 0.05, row and column subsampling at 0.8	300 trees
LSTM	Masking, LSTM(64), dropout 0.2, Dense(32), Dense(1); Adam at 10^{-3}	22,081 parameters
Chronos-2	Frozen backbone; head of Dense(128), dropout 0.2, Dense(64), dropout 0.2, Dense(1); Adam at 10^{-3}	119,477,664 frozen, 106,753 trained

The sizes in the final column are not comparable across families, for the reason given in Section 2.4.3. They indicate the scale of each model within its own family.

All four models are fitted using a squared-error criterion. Ordinary least squares minimises the residual sum of squares, XGBoost minimises a squared-error objective, and both neural models minimise mean squared error. None is trained directly against the asymmetric PHM score, and any difference between the ranking produced by that score and the rankings produced by the two symmetric metrics must be interpreted in that light.

6.5.3 Evaluation Protocol

Each model produces one prediction per test engine, at that engine's final recorded cycle, scored with the three metrics of Section 2.5. Predictions are clipped to the interval from zero to 125 under the piecewise target and bounded below at zero under the linear one. The evaluation reference remains the uncapped official remaining life, which reaches 145 cycles in FD001, so the upper bound follows from the target definition, not from a claim about physical remaining life. Section 7.4 applies both output settings to models fitted under both definitions, keeping the labels and the constraint on the predictions separable.

Three of the four models involve randomness in fitting: XGBoost subsamples rows and columns at each tree, and the two neural models initialise their weights at random. Each is fitted five times with different seeds, and the reported figure is the mean of the five run-level values with the standard deviation between them, since one run cannot separate a genuine difference from seed variation.

The significance analysis requires one value per model and engine. For each stochastic model the absolute error was computed per seed and engine and then averaged over the five seeds,

giving one error per model and test engine, on the same basis as the reported means. The analysis therefore rests on 100 engine-level pairs, not on 500 seed-engine combinations that are not independent. Whether a difference is more than incidental is assessed with the paired Wilcoxon signed-rank test of Section 2.6, which compares the two errors within each engine but does treat the engine-level pairs as independent. All six pairwise comparisons are performed, and the six raw p-values are adjusted by Holm's step-down procedure.

Computational cost is measured alongside accuracy. Three quantities are recorded: the time to fit a model once, the time to extract the Chronos-2 embeddings, incurred before any head is fitted, and the structural size in Table 6.4. The hardware is stated with each timing, since it differs: the tabular models were fitted on a CPU and the two neural models on a Tesla T4 GPU. The timings therefore describe the practical cost observed for each model rather than a controlled speed comparison.

6.6 Design Decisions and Final Configuration

Table 6.5 records the experiments and implementation checks that shaped the design. Three comparisons led to the rejection of a proposed addition or alternative, three identified problems that required correction, and the remainder established or documented retained choices. They are reported together because rejected alternatives and corrected errors form part of the methodological record.

Table 6.5. Experiments and checks, and the decisions arising from them.

Experiment	Outcome	Decision	Section
Initial four-model comparison	Ordering favoured the sequence pipelines	Observation horizon identified as uncontrolled	6.4.1
Target definition	Piecewise form supported by the early-life evidence and prior rationale	Piecewise adopted on empirical, structural, and literature grounds	6.3
Cap value	Comparison not neutral between caps	125 adopted on structural and literature grounds	6.3
Target scaling for the neural models	Unscaled LSTM target produced near-constant predictions	Neural targets divided by the applicable target maximum during fitting	6.3
Three levels of observed history	Large A-to-B gain; smaller and uncertain B-to-C gain	Full history adopted to equalise the observation horizon	6.4.1
Curvature and slope ratio	No improvement beyond fold variation	Five statistics retained	6.4.4
Length-matched batching for Chronos-2	Uniform padding gave the lower validation error	Uniform padding retained	6.4.4
Smoothing at three window lengths	No consistent improvement across the two tabular pipelines	Signals used as recorded	6.5.1
Early-stopping split	Row-based boundary could divide an engine	Engine-level inner split adopted; neural models retrained	6.2.2

Table 6.6 states the configuration each model is evaluated under in Chapter 7.

Table 6.6. Final configuration of the four models.

Model	History representation	Engines contributing to parameter fitting
Linear regression	Five statistics per sensor over the full observed history	100
XGBoost	Five statistics per sensor over the full observed history	100
LSTM	Post-padded sequence with masking	80
Chronos-2	Zero-pre-padded sequence, frozen embedding, mean pooling	90

All four use the piecewise target with a cap of 125, scaled during fitting for the two neural models (Section 6.3). Structural sizes are given in Table 6.4.

The comparisons reported in this chapter do not all carry the same evidential weight. The comparisons used to select the tabular history representation, assess the shape features, and decide on smoothing were obtained from the training engines under the validation protocol of Section 6.2.1. The cap-sensitivity analysis used the same protocol but was descriptive and did not determine the selected cap. One result is an explicit exception: the initial four-model comparison, conducted before the protocol was revised and reported as preliminary. Parameter counts and implementation checks are structural or procedural observations rather than validation results.

7 Results and Discussion

This chapter reports what the experiments produced and what the results mean. The four pipelines differ in more than one respect, so no single cause can be isolated. The official test engines were used after the pipeline had been frozen, for the two purposes recorded in Section 6.2.2.

7.1 Effect of the History Representation

The evidence for SQ1 comes from the training-only validation experiments of Section 6.4 (Tables 6.1 and 6.2). The first step produced the only large improvement, but it changed the observation horizon, the form of the representation and the number of inputs at once, so the gain cannot be traced to one factor. Extending the summary to the full history and adding the shape features changed the error by less than the variation between folds. The full history was kept to give every model family the same observation horizon, not because it predicted better.

The slope and the standard deviation describe a history in two different ways. The slope records the direction and the rate of change. The standard deviation records how much the signal moved, without keeping the order. Neither can be computed from a single reading, and the permutation results in Section 7.5 fit this: these two groups produced the largest rises in error.

The negative result for the shape features needs care. Section 5.4.2 found a steeper late-life slope in every training engine for twelve of the thirteen sensors, so the acceleration is present in the data. The experiment shows only that these two features did not help. They may repeat what the five statistics already carry, may be estimated too noisily over short histories, or may be unnecessary for a model that can already combine the existing inputs non-linearly. The experiment cannot separate these three.

7.2 Comparative Predictive Performance

SQ2 asks how the four pipelines compare when each one is given the full observed history through its assigned representation. The comparison is between complete pipelines, not between architectures alone.

Table 7.1 reports what each frozen pipeline achieved on the 100 test engines. For the three pipelines with randomness in fitting, every figure is a mean across five seeds; the standard deviation between those runs is given for the mean absolute error.

Table 7.1. Test performance of the four modelling pipelines. The figures are not comparable with the validation errors of Chapter 6, which scored engines at fixed distances from failure rather than at their final observed cycle.

Pipeline	Input	MAE, mean $\pm$ SD	RMSE	PHM score
Linear regression	Summary vector	14.51 (deterministic)	17.48	491.48
XGBoost	Summary vector	8.59 ± 0.08	12.03	245.37
LSTM	Padded sequence	9.71 ± 0.45	13.49	305.65
Chronos-2	Padded sequence	8.95 ± 0.40	12.08	239.04

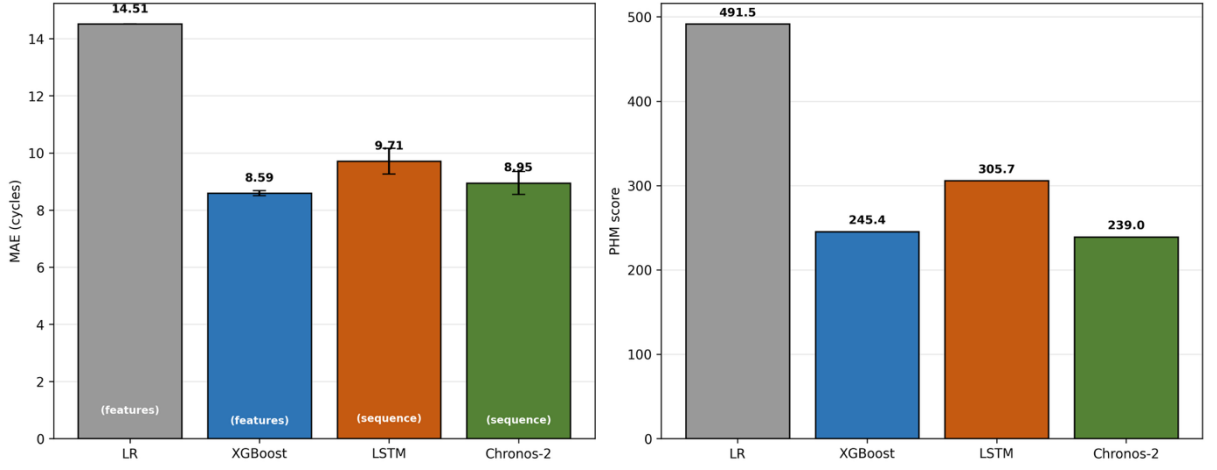


Figure 7.1. Performance of the four frozen modelling pipelines on the 100 FD001 test engines, by mean absolute error (left) and PHM score (right). Each pipeline keeps the same colour in both panels, and the label inside each bar records whether it receives summary features or an ordered sequence. Error bars give the standard deviation across five seed-specific runs for the three stochastic pipelines.

Figure 7.1 shows the same results as bars. Linear regression was the weakest pipeline on all three metrics. Its mean absolute error was at least 4.80 cycles higher than that of any other pipeline, and 5.92 cycles higher than the lowest value. Its PHM score was about twice the lowest score.

The other three pipelines were close together, within 1.12 cycles of one another in mean absolute error, and the two metrics do not rank them alike. XGBoost has the lowest mean absolute error, ahead of Chronos-2 by 0.36 cycles, while Chronos-2 has the lowest PHM score, ahead of XGBoost by 6.33. The LSTM is third on all three metrics. The reason the two disagree is taken up below.

Whether the differences among those three are more than incidental was tested with the paired Wilcoxon test described in Section 6.5.3. The test used one absolute error per pipeline and per engine, so each comparison rests on 100 paired values rather than on the 500 seed-level combinations, which are not independent. The results are given in Table 7.2.

Table 7.2. Pairwise Wilcoxon signed-rank tests on engine-level absolute errors. The median difference is the error of the first pipeline minus that of the second, so a positive value means the first was the less accurate.

Comparison	Median difference	p (raw)	p (Holm)	Significant at 5%
LR vs XGBoost	+6.06	<0.0001	<0.0001	Yes
LR vs LSTM	+5.32	<0.0001	0.0001	Yes
LR vs Chronos-2	+5.22	<0.0001	<0.0001	Yes
XGBoost vs LSTM	-1.06	0.0638	0.1915	No
XGBoost vs Chronos-2	-1.11	0.2171	0.4341	No
LSTM vs Chronos-2	+0.37	0.4151	0.4341	No

Linear regression differed from each of the other three after the Holm correction. None of the three comparisons among XGBoost, the LSTM and Chronos-2 reached the adjusted threshold.

This is an absence of evidence for a difference. It is not evidence that the three perform equally, because no equivalence test was run.

The test applies to paired absolute errors only, and it treats the engine-level pairs as independent. The PHM score could be split into per-engine contributions and tested the same way, but this was not done, so nothing here shows whether its differences are systematic.

Run-to-run variation belongs to the same question. Across the five seeds the two neural pipelines varied more between runs than the tree ensemble. The standard deviations are given in Table 7.1.

Fixed seeds did not give exact replication in this environment. A repeat of the LSTM experiment, with the same settings and the same seed values, gave an aggregate mean absolute error of 9.90 cycles instead of 9.71, with individual seeds differing by up to half a cycle. This difference is not contained in the standard deviation of 0.45, which was computed from a single set of five runs. It is reported separately as an observation about reproducibility.

The gap between linear regression and the other three has a structural cause. A single linear combination of the sixty-five inputs applies one coefficient to each input across its whole range. It cannot express a threshold above which the dispersion of a sensor starts to matter, or an interaction in which the slope of one sensor matters only when another is already high. The accelerating trajectories of Section 5.4.2 are consistent with such structure, although acceleration in the sensors does not by itself prove that the relationship between the summary statistics and remaining life is non-linear.

Section 7.4 supports this from another direction. Linear regression was far more sensitive than XGBoost to how the target was defined: under the same output constraint, changing the training labels reduced its error more than three times as much. Capping the early-life labels removes the large target values that pull a single global fit. The experiment does not separate this from other consequences of changing the response distribution.

One result narrows the explanation. The curvature and slope-ratio features did not improve linear regression either. The model therefore did not simply lack a description of sensor acceleration. Thresholds and interactions remain possible, and two extra trajectory descriptors cannot supply them to a model that stays linear in its inputs.

Three explanations fit the absence of a detected difference among XGBoost, the LSTM and Chronos-2, and the evidence does not choose between them.

The data may set a limit. Section 5.6 found that an engine's eventual lifetime could not be recovered from its earliest cycles, and that a usable signal appeared only after about eighty observed cycles. If the information available at a prediction point is bounded in this way, three pipelines with enough capacity would be expected to reach a similar level. That analysis used ridge regression on three statistics and predicted total lifetime rather than remaining life, so it does not set a ceiling for every model family.

The test may also be the limit, since a non-significant result can reflect a small effect, low power, or no difference at all, and no equivalence test was run to separate these. Or the representations may overlap: the five summary statistics may already carry part of the temporal

information that the sequence models learn from the raw histories. Section 7.5 is consistent with this but does not show that the statistics reproduce what the LSTM or Chronos-2 hold internally.

The reason the two error metrics rank the leading pipelines differently lies in how they are built, as Section 2.5 sets out. The mean absolute error depends only on the size of an error. The PHM score depends on its direction as well. A pipeline whose errors tend towards early prediction can therefore rank better on the PHM score than a pipeline with a similar mean absolute error but more late predictions.

The target experiment shows how large this difference in sensitivity can be. Clipping the stored predictions of the linear-target XGBoost model, without refitting it, reduced its mean absolute error by 34.4 per cent and its PHM score by 94.6 per cent. For linear regression the reductions were 21.5 and 77.3 per cent. Because the penalty grows exponentially, removing a few large positive errors moves the score far more than it moves the mean absolute error. This explains how two metrics can order the pipelines differently, but it does not identify which engines produced the gap between Chronos-2 and XGBoost; that would need the individual score contributions.

Which metric should count more cannot be settled by a statistical test; it depends on the cost of each direction of error. Where overestimating remaining life delays maintenance and carries the greater risk, the PHM score is closer to the operational objective. Where an error of a given size costs the same either way, the mean absolute error is the more direct measure. Its exponential penalties are one assumed cost structure, not costs established for every maintenance setting.

7.3 Computational Cost and Inspectability

Where accuracy does not separate three of the pipelines, the resources each one requires become relevant to a choice between them. SQ3 asks what each approach costs. Table 7.3 records what each one required.

Table 7.3. Measured computational cost. Linear regression was fitted in under a second and was not timed precisely.

Pipeline	Mean fitting time per run	Computation reused across runs	Hardware
Linear regression	< 1 s	None	CPU
XGBoost	22.6 s	None	CPU
LSTM	69 s	None	Tesla T4
Chronos-2	24 s for the regression head	566 s for embedding extraction	Tesla T4

The times are means across the five seed-specific runs, except for the embedding extraction, which was performed once. They cover the fitting step alone and, for Chronos-2, the extraction of embeddings from prepared sequences; data loading, preprocessing, feature construction, prediction, metric calculation and inference are not included. The two model families also ran on different hardware. The figures are therefore neither end-to-end timings nor ratios of one another. XGBoost, the LSTM and Chronos-2 differ considerably in the work they require, yet no difference in paired absolute error was detected between them.

XGBoost was shown to fit on the available CPU in seconds. The LSTM was attempted on a laptop CPU in the preliminary work and stopped after more than eleven hours without finishing, which shows that this configuration was impractical on that machine but gives no fitting time. Chronos-2 was never attempted on a CPU. The study therefore records different observed execution environments, not an absolute hardware requirement.

The embedding cost for Chronos-2 behaves differently from fitting. Once embeddings have been computed for a fixed set of histories, they are reused across seeds and across any retuning of the head; new or extended histories need embeddings for the affected engines only. The cost is moved away from repeated fitting rather than removed.

Section 7.5 examines the XGBoost pipeline with a permutation analysis that connects its predictions to named statistics and sensors, and no equivalent analysis was carried out for the two sequence pipelines. This is a gap in what the study produced rather than proof that sequence models cannot be examined; methods for doing so exist.

Under the resources and analyses used here, computational cost and inspection evidence both favour XGBoost. This is not a general advantage: establishing one would require the measurements this work did not make, namely feature construction, inference latency, memory and energy.

7.4 The Target Definition and the Output Constraint

SQ4 asks how the construction of the target affects performance. The original comparison changed two things at once, and separating them changes the answer.

A piecewise pipeline differs from a linear one in its training labels and in what is done with a prediction after it is made. The piecewise pipeline restricts its output to the range of the capped label. The linear pipeline does not, because that would discard the larger values the linear form exists to allow. Comparing the two as they stand mixes the labels with the post-processing.

The two factors were therefore separated. Linear regression was fitted once under each target definition, and XGBoost once per definition and prescribed seed. For every fitted model the unconstrained predictions were stored, and both output settings were then applied to the same stored predictions without refitting. The representation, hyperparameters, training data, test engines and seed protocol were identical across the two definitions. All four combinations were evaluated against the same unrestricted official test values, of which eleven exceed 125 cycles. Table 7.4 reports all four combinations.

Table 7.4. Test performance under both training targets and both output settings.

Pipeline	Training target	Output setting	MAE, mean $\pm$ SD	RMSE	PHM score
Linear regression	Linear	Lower bound at 0	25.07 (det.)	30.88	9,843.28
Linear regression	Linear	Clipped to [0, 125]	19.67 (det.)	24.22	2,234.44
Linear regression	Piecewise	Lower bound at 0	14.59 (det.)	17.59	499.94
Linear regression	Piecewise	Clipped to [0, 125]	14.51 (det.)	17.48	491.48
XGBoost	Linear	Lower bound at 0	15.37 $\pm$ 0.32	23.50	8,866.26

Pipeline	Training target	Output setting	MAE, mean $\pm$ SD	RMSE	PHM score
XGBoost	Linear	Clipped to [0, 125]	10.09 $\pm$ 0.31	14.36	477.69
XGBoost	Piecewise	Lower bound at 0	8.58 $\pm$ 0.07	12.01	245.09
XGBoost	Piecewise	Clipped to [0, 125]	8.59 $\pm$ 0.08	12.03	245.37

With the training labels held fixed, clipping the predictions to [0, 125] reduced the error of both tabular pipelines by over five cycles under the linear target, without fitting anything again. Under the piecewise target the same operation changed the error by less than a tenth of a cycle for either pipeline.

With the output setting held fixed at [0, 125], changing from linear to piecewise labels reduced the XGBoost error by 1.50 cycles and the linear-regression error by 5.16. With the lower bound alone, the corresponding reductions are 6.79 and 10.48 cycles. Both are valid label contrasts, each measured at a fixed output setting.

The two contrasts differ by 5.29 cycles, more than fifteen times the seed-to-seed spread of any cell, though the interaction was not formally tested. The label contrast is more than four times larger without the constraint than with it, and the constraint behaves the same way in reverse: over five cycles under linear labels and almost nothing under piecewise ones. Neither factor has a single value that holds in every context.

The PHM score shows the interaction more sharply. Under the linear target, clipping cut the XGBoost score by more than an order of magnitude; under the piecewise target it changed it by less than one point. The metric applies an exponential penalty to positive errors, so a small number of large positive deviations can dominate the total. Identifying the engines responsible would require the individual score contributions, which were not examined.

XGBoost was the less sensitive of the two models once both were subject to the same constraint. This may follow from its greater flexibility in mapping the summary statistics onto either form of the target, while a single global linear fit remains more sensitive to the changed distribution of the response and to the reduced influence of large early-life labels. The experiment was not designed to separate this explanation from other differences between the two fitting procedures.

A practical point follows. A study that changes the target definition and the post-processing together cannot say which of the two produced the improvement: what a target definition appears to be worth depends on what else changes with it. Guilherme reported the target formulation as the largest single factor on FD001 [20], and the direction of the present contrasts agrees.

The comparison covers the two tabular pipelines only. And the constraint is not neutral: for the eleven test engines whose remaining life exceeds 125 cycles, a prediction restricted to [0, 125] cannot be exact. This does not mean that clipping increased the error for those engines, since that depends on where the unconstrained prediction lay, but the aggregate improvement should not be read as an improvement for every engine.

7.5 Post-hoc Inspection and the Behaviour of Sensor s9

This section asks a narrower question about the best-performing tabular pipeline: which of the sixty-five inputs does the fitted model rely on? The question is answered by the permutation importance of Section 2.4.2, applied here to the test engines. Predictions were clipped to [0, 125] first, as in the final pipeline, and each group was permuted jointly, with one shared row ordering per repetition. The values are means over ten repetitions on one fitted model at seed 42, whose baseline error was 8.47 cycles against the five-seed mean of 8.59. The spreads describe variation across the ten permutations, not across engines or seeds. Table 7.5 gives the result for the five summary statistics.

Table 7.5. Importance of each summary statistic, all thirteen columns permuted jointly.

Statistic	Rise in test MAE (cycles)
Standard deviation	16.51 ± 2.04
Slope	11.99 ± 1.40
Mean	1.87 ± 0.44
Last value	1.64 ± 0.30
Delta	0.32 ± 0.16

The standard deviation and slope groups produced by far the largest increases, and the mean, last-value and delta groups all stayed below two cycles. Because the values are not additive, they are not summed here or read as shares of a total. The last-value group, the single reading the tabular models received under representation A of Section 6.4.1, is among the smallest. Table 7.6 repeats the analysis by sensor.

Table 7.6. The six highest-ranked sensor groups, all five summaries permuted jointly.

Sensor	Rise in test MAE (cycles)
s11	7.17 ± 0.95
s9	5.10 ± 0.67
s4	3.89 ± 0.57
s20	2.92 ± 0.47
s12	1.67 ± 0.32
s15	1.31 ± 0.25

Figure 7.2 shows both views together. Sensor s11 produced the largest sensor-group importance. Among the individual inputs belonging to it, the standard deviation of s11 produced the largest single effect, raising the error by 5.86 cycles. The next largest individual effects were 2.08 cycles for the standard deviation of s9 and 1.55 for the slope of s20.

Because the importances were computed on the official test engines, the analysis is descriptive and post-hoc. It was not used for feature selection, for tuning, or as evidence for SQ5. The analysis uses one fitted model, so its stability across fitted instances was not assessed.

Correlated inputs can substitute for one another, so a small importance does not establish that an input is uninformative in isolation. And joint permutation preserves the relationships among columns within a permuted group while breaking their relationships with the unpermuted ones, which can produce combinations that do not occur in the original engines.

Sensor s9 ranked second, and the results concerning it appear at first to conflict. Section 5.4.1 found that only 71 of the 100 training engines agreed on the direction of its trend, against complete agreement for the other twelve sensors. Section 5.4.3 found that its spread across engines widened towards failure, where ten of the thirteen sensors narrowed.

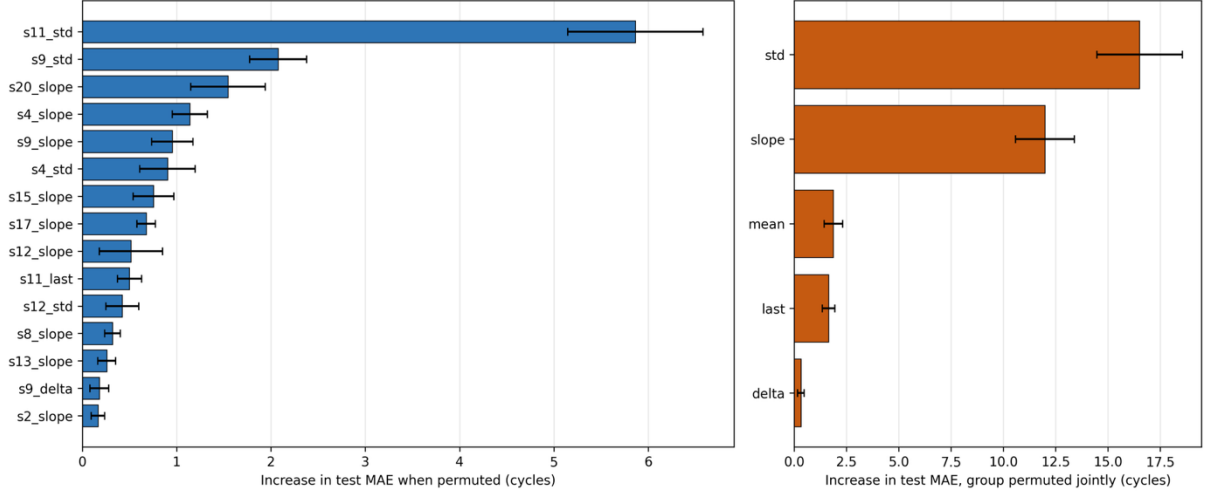


Figure 7.2. Post-hoc permutation importance for the seed-42 XGBoost pipeline on the 100 FD001 test engines, measured as the rise in mean absolute error when an input is shuffled. The left panel gives the fifteen largest individual-input effects and the right panel the five summary-statistic groups, each permuted jointly across all thirteen sensors. Error bars give the standard deviation across ten permutations.

The conflict disappears once the three measurements are kept apart. Trend agreement counts how many engines move in the same direction. The spread ratio compares the variation across engines at the end of life with the variation at the start. The permutation effect measures something else: the five s9 summaries are computed inside each engine's own history, but their group effect comes from shuffling those summaries across test engines. The model therefore relied on differences between engines in a statistic computed within each of them, and an inconsistent direction does not prevent that.

What the sensor measures gives some physical context. Sensor s9 records the physical core speed, and FD001 represents degradation of the high-pressure compressor, which sits on the core spool. C-MAPSS simulates the engine together with a closed-loop control system [3], so core speed can reflect both the state of the engine and the response of the controller to it. The published data do not report controller states, so the behaviour cannot be attributed to controller action.

Differences in the size and the variability of the s9 trajectories may be informative even though their directions are less consistent. That would let descriptive irregularity and predictive reliance coexist, but it would not show that the irregularity produced the importance: a sensor with a consistent direction can still differ between engines in level or dispersion. Testing this would require the injected health parameters or controlled interventions in the simulator, which the published dataset does not provide.

The descriptive results come from the training engines and the permutation result from the test engines, using one fitted model; neither confirms the other. And the importance is conditional, since it describes reliance given the other inputs available to the model.

The case matters beyond the sensor itself. A screening rule based only on directional consistency would have removed s9, and the permutation analysis suggests that this would have dropped a group the model drew on heavily. It does not show that refitting without s9 would raise the error by that amount, since the remaining inputs might compensate. The preprocessing actually applied did not treat irregular behaviour as grounds for exclusion (Section 5.3). The episode shows the risk of turning one descriptive property into a sufficient screening rule.

7.6 Comparison with Published Work

Chapter 3 identified three gaps in the FD001 literature, and this section returns to them. The root mean squared errors of the XGBoost and Chronos-2 pipelines fall inside the range of 10.77 to 16.14 cycles reported by the studies kept in Section 3.6, so the results sit on the same scale. They cannot be ranked against any single published method, because those studies differ in target construction, observation horizon, preprocessing, configuration selection and averaging across runs.

The first gap was the absence of a directly comparable FD001 result for a tabular pipeline. This work supplies one: the tabular and sequence pipelines receive the same observation horizon at the prediction point, each through a representation suited to its family. Tree ensembles had been used on this subset before; what is new is a measurement made under an aligned history.

The second gap was reporting. Accuracy is covered far more consistently in this literature than metric coverage, cost or model inspection. All three metrics appear here, with stage-specific execution times, structural sizes and a post-hoc inspection of one fitted pipeline. This is not a standardised efficiency benchmark, for the reason given in Section 7.3.

The third gap was the form in which a history is presented, never compared across model families. It is narrowed, not closed: the observation horizon is aligned across the four pipelines, but each representation stays tied to its family (Section 6.4). Within the tabular family, changing the representation of the same history altered performance materially (Section 7.1); across families, no adjusted difference in paired absolute error was found among the three leading pipelines (Section 7.2). Input construction and protocol are likely to contribute to the spread across published results. How much of that spread reflects representation rather than architecture stays open, since the two were neither crossed here nor held constant in the reviewed studies. The qualification of Section 3.1 applies to these results as much as to the published ones: one protocol, one subset. Section 7.8 states what that limits.

7.7 Answers to the Research Questions

The five subsidiary questions of Section 1.3 are answered as follows; the evidence and its qualifications are given in the sections cited.

SQ1. Summarising the observed history, instead of supplying the prediction cycle alone, lowered the XGBoost validation error by forty per cent; extending the summary to the full

history and adding descriptions of trajectory shape changed nothing beyond the variation between folds (Section 7.1).

SQ2. Linear regression differed from each of the other three pipelines after Holm adjustment, and no adjusted difference was detected among XGBoost, the LSTM and Chronos-2, an absence of evidence, not evidence of equality (Section 7.2).

SQ3. XGBoost was fitted on a CPU in under half a minute; both sequence pipelines ran on a GPU, and Chronos-2 additionally required an embedding stage that dominates its cost. Because the hardware and the timed stages differ, these figures are not speed ratios (Section 7.3).

SQ4. The training labels and the output constraint do not act independently: for XGBoost the effect of changing the labels was more than four times larger without the constraint than with it, so neither factor has one context-free effect (Section 7.4).

SQ5. Ten variables carried no usable variation and s14 duplicated s9, leaving thirteen sensors; extreme values concentrate close to failure and were therefore retained; and no usable early-life signal was recovered before about eighty observed cycles (Sections 5.3, 5.5 and 5.6).

7.8 Limitations and the Scope of the Conclusions

The limitations of this work have been stated where they arose. Table 7.7 collects them with the consequence of each.

Table 7.7. Limitations and their consequences.

Limitation	Consequence	Section
The length-matched batching comparison changed several factors at once	It establishes that the alternative pipeline performed worse as a whole, not which change caused it	6.4.4
Only FD001 was studied	Nothing here extends to the subsets with several operating conditions or fault modes	1.4
C-MAPSS provides simulated trajectories rather than observations from an operating fleet	The results do not establish performance under real sensor faults, maintenance interventions, or deployment-related distribution shift	1.4, 4.1
No equivalence test or power analysis was performed	The comparisons that did not reach significance do not establish equal performance	7.2
Timings were obtained on different hardware and cover different stages	Runtimes cannot be compared as ratios across model families	7.3
Inference latency, throughput, and memory use were not measured	The cost figures describe selected offline stages, not the cost of operating a pipeline in service	7.3
Chronos-2 receives zero padding without an explicit mask	The embeddings may encode padding, or the length of the history, alongside the recorded trajectory	6.4.3
The sequence pipelines reserve engines for early stopping	The LSTM is fitted on eighty engines and the Chronos-2 head on ninety, against one hundred for the tabular pipelines	6.2.1
Three method-level choices were first compared on the test set before the protocol was revised	The comparisons were re-run on training-only validation, but the reported test figures may retain some optimism	6.2.2

Limitation	Consequence	Section
Configurations were selected on the raw-value representation and then held fixed	The representation comparison measures performance under one common configuration, not the best achievable after tuning each representation separately	6.2.2
The sequence pipelines were not refitted under the linear target	The target comparison applies to the tabular pipelines only	7.4
The permutation analysis used one fitted model and the test engines	Its ranking is descriptive and its stability across fits was not assessed	7.5
Five seeds were used	The reported spread is estimated from a small number of runs	7.2
The Chronos-2 backbone was used frozen and was not fine-tuned	The results describe what a frozen pretrained representation transfers, not what the architecture could reach if adapted to turbofan data	6.4.3
Chronos-2 was not attempted on a CPU	The results establish different observed execution environments, not an absolute hardware requirement	7.3

Three of these bound the argument most heavily: the unmasked padding supplied to Chronos-2 (Section 6.4.3), the restriction to FD001, and the unequal fitting sets, which leave the four pipelines fitted on eighty, ninety and one hundred engines. None of them alters the recorded outputs of the experiments; they alter what those outputs can be taken to support.

8 Conclusion and Outlook

Section 7.7 answers the five subsidiary questions in detail. Taken together, the results support moving beyond a single global linear model on this benchmark, but they do not establish an accuracy benefit from replacing the tabular tree ensemble with a sequence model.

8.1 Contributions

Three contributions follow, each bounded by the limitations of Section 7.8. The comparison runs under a matched observation horizon, which no study reviewed in Section 3.6 does across model families, and it places a tabular pipeline under the same dataset, target definition, output constraint, test engines and metrics as the sequence pipelines; this removes one source of incomparability, though it does not separate architecture from representation (Section 7.6). The training labels are also separated from the output constraint, since the factorial experiment of Section 7.4 treats them as two design choices rather than one, which is what makes each of them interpretable. And rejected or revised design decisions are documented rather than omitted: Table 6.5 lists what was tested, what was rejected and what was corrected, and Section 6.6 keeps the three categories apart.

Two practical implications follow. Where accuracy cannot separate pipelines, the choice falls to computational workload and to how far each can be inspected (Section 7.3). An exclusion rule should also be tested against predictive performance before it is applied, since a screening criterion based on directional consistency would have removed an input the fitted model relied on heavily (Section 7.5).

8.2 Future Work

The limitations of Section 7.8 point to work that would extend what is reported here, ordered by how directly each bears on the conclusions. Evaluation on independent data would test the frozen pipeline on genuinely new engines. Repeating the whole development procedure under nested engine-level resampling, with every selection step confined to the inner folds, would instead give a protected estimate of the procedure itself; the two are not interchangeable. Aligning the fitting sets would remove a further difference from the comparison, through the refitting procedure set out in Section 6.2.1 that this work did not carry out.

Crossing representation with model family is the most direct test of whether the absence of a detected difference among the three leading pipelines reflects the architectures or what each received: within a common observation window, each family could receive either ordered values or summary features, with the training budget and selection procedure held fixed. A padding-aware input for Chronos-2 would settle whether its representation encodes padding or history length, through an explicit attention mask where the interface supports one, or direct handling of variable-length input. A test against a margin fixed in advance would allow a positive statement of similarity among the three leading pipelines, rather than the absence of a detected difference reported here. Fitting with and without s9 across seeds would estimate what excluding that sensor costs.

Fine-tuning the pretrained backbone would measure what adaptation adds, and isolating the value of the pretraining itself would require a third arm: the same architecture trained from

random initialisation under a matched budget. Extension to FD002 and FD004 would test whether the conclusions persist where operating conditions and fault modes vary, and two measurement gaps remain alongside it, since inference latency, throughput, peak memory and feature construction were not timed and only the tabular pipeline was inspected.

What this work suggests, in the end, is that a comparison of model families depends on more than the architectures compared: how a history is represented, which metric is reported, how the target is defined and what constraint is applied to the predictions all shaped what the results could support, and a comparison of complete pipelines cannot attribute a difference to architecture unless the other components are controlled separately.

Appendix A Sensor and Operational-Setting Reference

This appendix reproduces the physical description of the twenty-four candidate variables of the FD001 dataset referred to in Section 4.4.

Table A.1. Physical description of the three operational settings and twenty-one sensors of the FD001 dataset. Engine modules: LPC, HPC, HPT, LPT. Source of symbols, descriptions, and units: [3].

ID	Symbol	Physical meaning	Module / location	Unit	Data type
op1	—	Operational setting 1 (related to altitude)	Flight condition	—	continuous
op2	—	Operational setting 2 (related to Mach number)	Flight condition	—	continuous
op3	—	Operational setting 3 (related to throttle-resolver angle)	Flight condition	—	continuous
s1	T_2	Total temperature at fan inlet	Fan inlet	°R	continuous
s2	T_{24}	Total temperature at LPC outlet	LPC	°R	continuous
s3	T_{30}	Total temperature at HPC outlet	HPC	°R	continuous
s4	T_{50}	Total temperature at LPT outlet	LPT	°R	continuous
s5	P_2	Pressure at fan inlet	Fan inlet	psia	continuous
s6	P_{15}	Total pressure in the bypass duct	Bypass duct	psia	continuous
s7	P_{30}	Total pressure at HPC outlet	HPC	psia	continuous
s8	N_f	Physical fan speed	Fan (low-pressure spool)	rpm	continuous
s9	N_c	Physical core speed	Core (high-pressure spool)	rpm	continuous
s10	epr	Engine pressure ratio (P_{50}/P_2)	Overall engine	—	continuous
s11	P_{s30}	Static pressure at HPC outlet	HPC	psia	continuous
s12	ϕ	Ratio of fuel flow to P_{s30}	HPC / fuel system	pps/psi	continuous
s13	NR_f	Corrected fan speed	Fan (low-pressure spool)	rpm	continuous
s14	NR_c	Corrected core speed	Core (high-pressure spool)	rpm	continuous
s15	BPR	Bypass ratio	Overall engine	—	continuous
s16	far_B	Burner fuel-air ratio	Combustor	—	continuous
s17	htBleed	Bleed enthalpy	Bleed system	—	continuous
s18	$N_{f,dmd}$	Demanded fan speed	Control system (commanded)	rpm	continuous
s19	$PCNfR_{dmd}$	Demanded corrected fan speed	Control system (commanded)	rpm	continuous

ID	Symbol	Physical meaning	Module / location	Unit	Data type
s20	W_{31}	HPT coolant bleed flow	HPT cooling	lbm/s	continuous
s21	W_{32}	LPT coolant bleed flow	LPT cooling	lbm/s	continuous

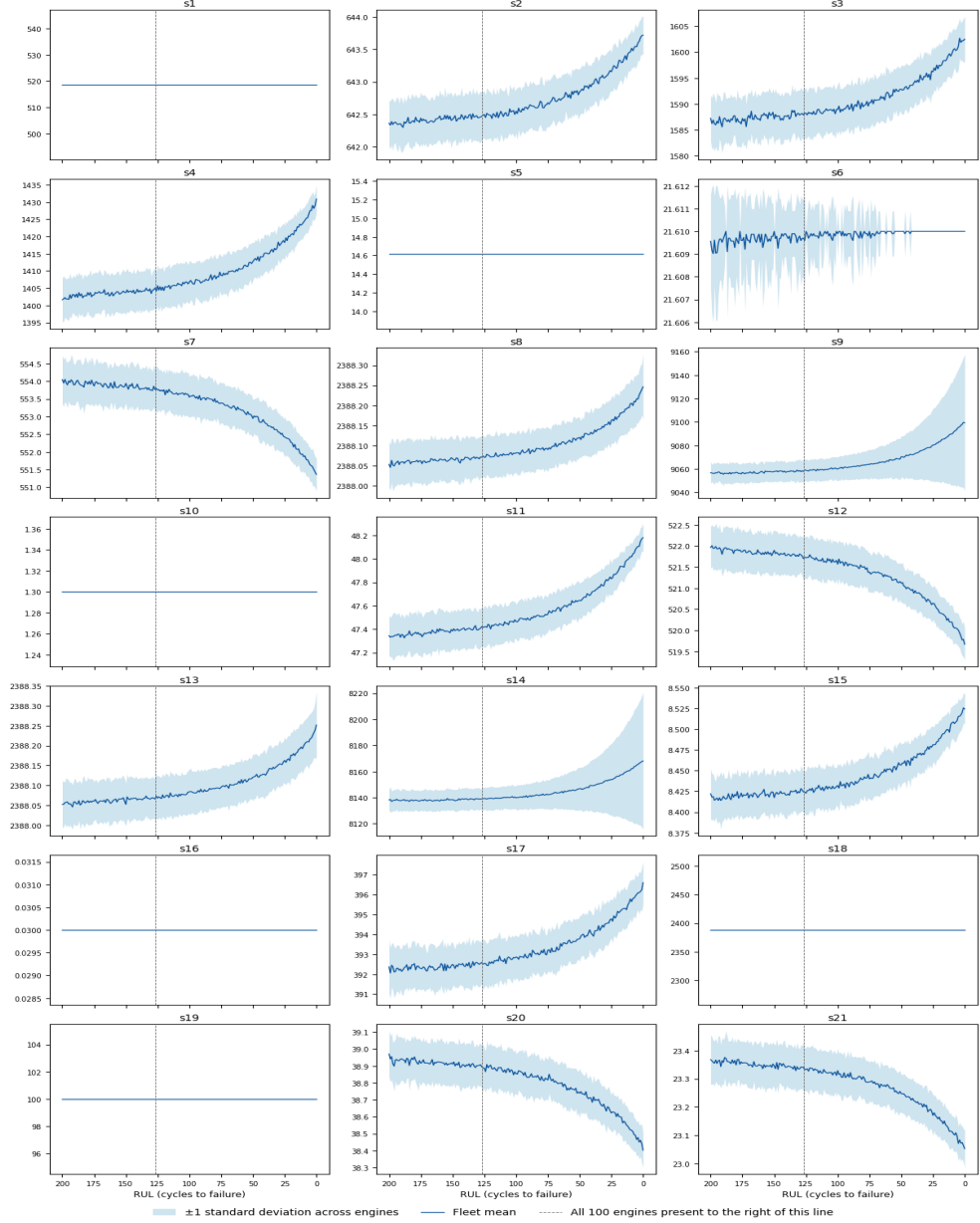


Figure A.1. Mean recorded value of each of the twenty-one sensors across the FD001 training engines, aligned by remaining useful life over the final 200 cycles before failure. The shaded band gives one standard deviation across engines. All 100 engines contribute to the right of the dashed line at a remaining life of 127 cycles; to its left only the engines surviving that long are represented. The seven channels removed in Section 5.3 appear as flat lines.

Appendix B Data-Quality and Data-Separation Checks

This appendix collects the verification checks referred to in Sections 5.1, 5.5 and 6.2.2. Both tables record checks carried out on the data and on the experimental pipeline; their outcomes are stated in the corresponding sections of the main text.

Table B.1. Integrity checks and resulting decisions.

Check	Train	Test	Decision
Engines	100	100	No action
Rows	20,631	13,096	Expected recording asymmetry
Missing values	0	0	No imputation
Fully duplicated rows	0	0	No row removal
Duplicated (unit, cycle) pairs	0	0	No row removal
Non-numeric columns	0	0	No type conversion

Table B.2. Points of exposure between training and test data, and their status.

Point of exposure	Implemented check	Status
Engine partitioning	No unit appears in both the fitting and validation sets	Verified
Early stopping	Explicit engine-level validation set	Corrected and rerun
Sensor scaling	Scaler fitted after each fold split, on fitting engines only	Verified
Embedding standardisation	Fitted on the embeddings of fitting engines only	Verified
Input construction	No input cycle later than the prediction cycle	Verified
Padded sequence length	Fixed once from the official training set	Prespecified exception
Test set	Preliminary uses recorded separately from the confirmatory evaluation	Recorded in Section 6.2.2

Continuity of the sensor trajectories. The check referred to in Section 5.5 used sensor s11, which records the static pressure at the outlet of the high-pressure compressor, the module through which FD001 degrades. The analysis was not extended to the other twelve retained sensors.

Three quantities were computed for each of the 100 training engines on a smoothed version of the signal, smoothing being a rolling average that suppresses cycle-to-cycle fluctuation so that the underlying movement becomes visible. The net rise is the difference between the smoothed value at the end of the trajectory and at its beginning. The maximum drawdown is the largest fall from any running peak; a sustained restoration would produce a drawdown comparable in size to the overall rise. The third quantity is the proportion of cycles on which the smoothed value increases, for which a value near one half would indicate no directional tendency. An engine was flagged if either its net rise was negative or its maximum drawdown exceeded its own net rise. Both conditions were fixed before the analysis was run.

The third quantity depends on the degree of smoothing, and both settings examined are reported. With a five-cycle window the proportion averages approximately 0.55, close to the no-tendency value; with a twenty-cycle window it averages 0.661. The difference follows from the magnitudes involved: the cycle-to-cycle variation is of the order of 0.1 psia whereas the per-cycle contribution of the trend is of the order of 0.005, so a five-cycle average remains dominated by the former. The twenty-cycle window also sets the resolution of the test, since a restoration shorter than it would be averaged away.

The net rise is positive for all 100 engines, with a mean of 0.678 psia and a range from 0.447 to 0.959. The maximum drawdown averages 0.046 psia, about seven per cent of the mean net rise, and the largest in any single engine is 0.110. The proportion of increasing cycles averages 0.661 with a minimum of 0.580. The mean net rise of 0.678 psia differs from the mean change of 0.796 reported in Section 5.4.3 because the two are defined differently: the present figure comes from a signal smoothed over twenty cycles, which draws the endpoints inward, whereas the earlier one compares the means of the first and last five raw cycles.

References

- [1] M. Mołęda, B. Małysiak-Mrozek, W. Ding, V. Sunderam, and D. Mrozek, "From corrective to predictive maintenance: A review of maintenance approaches for the power industry," *Sensors*, vol. 23, no. 13, Art. no. 5970, 2023, doi: 10.3390/s23135970.
- [2] M. Fernandes, J. M. Corchado, and G. Marreiros, "Machine learning techniques applied to mechanical fault diagnosis and fault prognosis in the context of real industrial manufacturing use-cases: A systematic literature review," *Applied Intelligence*, vol. 52, pp. 14246–14280, 2022, doi: 10.1007/s10489-022-03344-3.
- [3] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. Int. Conf. Prognostics and Health Management*, Denver, CO, USA, 2008, pp. 1–9, doi: 10.1109/PHM.2008.4711414.
- [4] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in *Proc. Int. Conf. Prognostics and Health Management*, Denver, CO, USA, 2008, pp. 1–6, doi: 10.1109/PHM.2008.4711422.
- [5] C. Ferreira and G. Gonçalves, "Remaining useful life prediction and challenges: A literature review on the use of machine learning methods," *Journal of Manufacturing Systems*, vol. 63, pp. 550–562, 2022, doi: 10.1016/j.jmsy.2022.05.010.
- [6] O. Asif, S. A. Haider, S. R. Naqvi, J. F. W. Zaki, K.-S. Kwak, and S. M. R. Islam, "A deep learning model for remaining useful life prediction of aircraft turbofan engine on C-MAPSS dataset," *IEEE Access*, vol. 10, pp. 95425–95440, 2022, doi: 10.1109/ACCESS.2022.3203406.
- [7] P. P. Walsh and P. Fletcher, *Gas Turbine Performance*, 2nd ed. Oxford, U.K.: Blackwell Science, 2004.
- [8] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *Proc. IEEE Int. Conf. Prognostics and Health Management*, Dallas, TX, USA, 2017, pp. 88–95, doi: 10.1109/ICPHM.2017.7998311.
- [9] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," *Reliability Engineering & System Safety*, vol. 172, pp. 1–11, 2018, doi: 10.1016/j.ress.2017.11.021.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [11] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.
- [12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

- [14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, Long Beach, CA, USA, 2017, pp. 5998–6008.
- [16] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. P. Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, and Y. Wang, "Chronos: Learning the language of time series," *Transactions on Machine Learning Research*, 2024.
- [17] A. F. Ansari et al., "Chronos-2: From univariate to universal forecasting," 2025, arXiv:2510.15821, doi: 10.48550/arXiv.2510.15821.
- [18] G. A. G. G. Felício, "Predicting remaining useful life with machine learning algorithms for predictive maintenance," M.Sc. thesis, Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal, 2021. [Online]. Available: <https://fenix.tecnico.ulisboa.pt/cursos/memec/dissertacao/846778572213646>
- [19] U. Thakkar and H. Chaoui, "Remaining useful life prediction of an aircraft turbofan engine using deep layer recurrent neural networks," *Actuators*, vol. 11, no. 3, Art. no. 67, 2022, doi: 10.3390/act11030067.
- [20] D. N. V. B. Guilherme, "Remaining useful life prediction on the NASA CMAPSS dataset comparing LSTM and transformer models," M.Sc. dissertation, Instituto Superior de Engenharia do Porto, Politécnico do Porto, Porto, Portugal, 2025. [Online]. Available: <http://hdl.handle.net/10400.22/30859>
- [21] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning with Applications in Python*. Cham, Switzerland: Springer, 2023.
- [22] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000, doi: 10.1162/089976600300015015.
- [23] Y. Mo, Q. Wu, X. Li, and B. Huang, "Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit," *Journal of Intelligent Manufacturing*, vol. 32, no. 7, pp. 1997–2006, 2021, doi: 10.1007/s10845-021-01750-x.
- [24] J. Li, K. Wang, X. Hou, D. Lan, Y. Wu, H. Wang, L. Liu, and S. Mumtaz, "A dual-scale transformer-based remaining useful life prediction model in Industrial Internet of Things," *IEEE Internet of Things Journal*, vol. 11, no. 16, pp. 26656–26667, 2024, doi: 10.1109/JIOT.2024.3376706.
- [25] M. Ma, Z. Wang, and Z. Zhong, "Transformer encoder enhanced by an adaptive graph convolutional neural network for prediction of aero-engines' remaining useful life," *Aerospace*, vol. 11, no. 4, Art. no. 289, 2024, doi: 10.3390/aerospace11040289.

- [26] U. Yıldırım and H. Afşer, "Linear methods for predictive maintenance: The case of NASA C-MAPSS datasets," *Applied Sciences*, vol. 15, no. 18, Art. no. 9945, 2025, doi: 10.3390/app15189945.
- [27] A. L. Ellefsen, E. Bjørlykhaug, V. Æsøy, S. Ushakov, and H. Zhang, "Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture," *Reliability Engineering & System Safety*, vol. 183, pp. 240–251, 2019, doi: 10.1016/j.ress.2018.11.027.
- [28] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [29] D. C. Montgomery and G. C. Runger, *Applied Statistics and Probability for Engineers*, 7th ed. Hoboken, NJ, USA: Wiley, 2018.